Meetup ElasticFR #102

16 Avril 2026

LA HOME SWEET COMPANY
WE ARE SPECIALISTS



18 / 19 Place des Reflets
92400 COURBEVOIE

elastic
The Search AI Company

Chaps Vision

elastic

Gestion de la mémoire d'un agent conversationnel avec Elasticsearch

© ChapsVision - All rights reserved

Chaps Vision



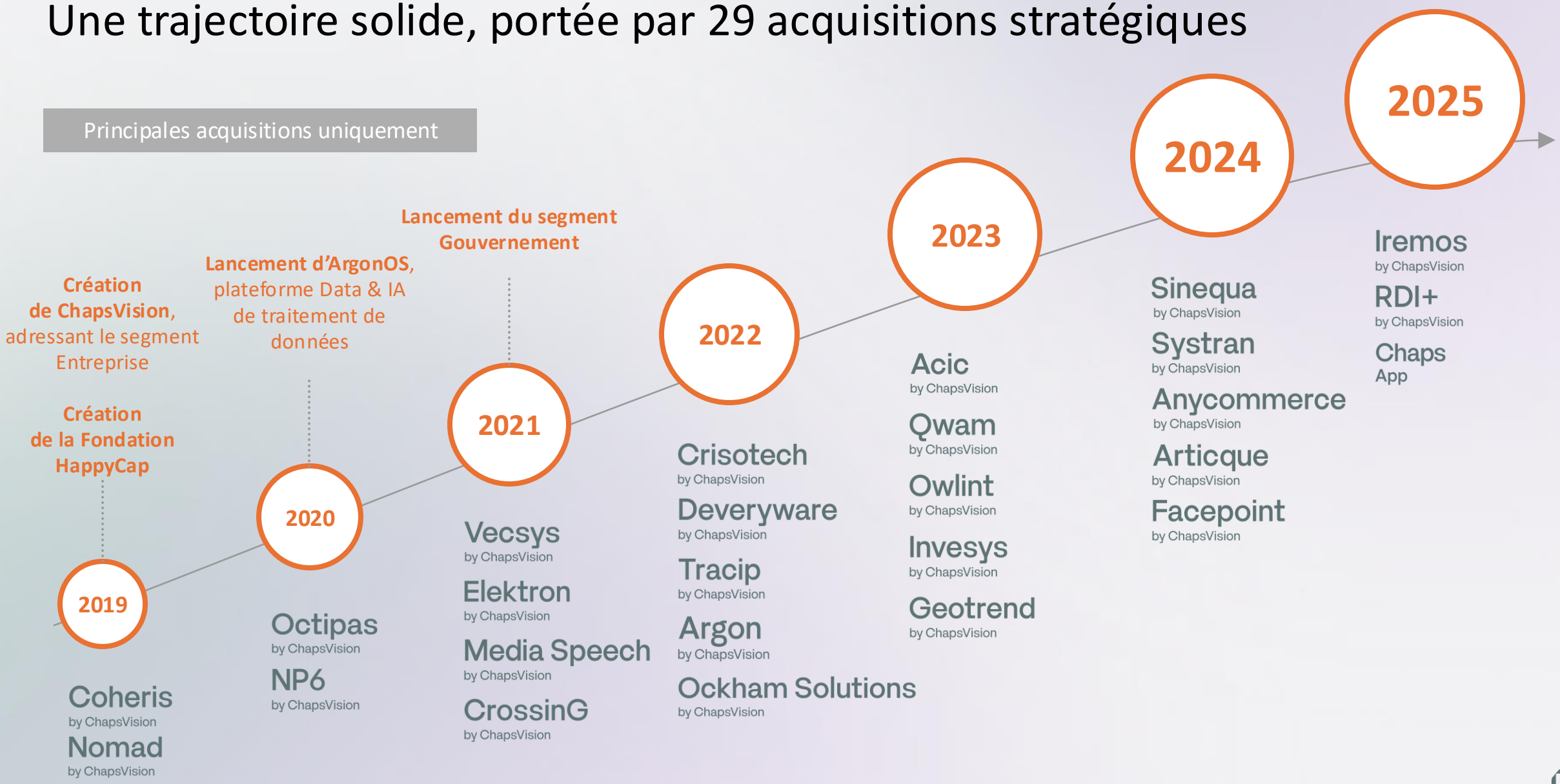
Louis Jourdain



Ivan Monnier

Une trajectoire solide, portée par 29 acquisitions stratégiques

Principales acquisitions uniquement



© ChapsVision - All rights reserved

ChapsVision en chiffres

~200 M€
de chiffre d'affaires

350+ M€
investis dans l'entreprise

29
acquisitions réalisées depuis le lancement

1,000+
collaborateurs dans le monde entier

2,000+
clients grands comptes B2B ou B2G

40+
pays dans lesquels nos solutions sont utilisées

Entreprise + Régalien

© ChapsVision - All rights reserved

Le Chapter IA

→ Les "grandes" solutions du groupe ont déjà une démarche IA agentique

Elles n'ont pas besoin de nous.

→ Membres de la Squad Text Analytics

Avec la Squad Audio , la Squad Vision et la Squad IA Ops

→ Fournir des "Capabilities IA" pour les "petites" solutions du groupe

Sous forme d'une API NLP

→ Produit interne

Destiné à être utilisé par les "petites" solutions dans leur distribution Cloud ou On premise

© ChapsVision - All rights reserved

-1

Le bon temps

Avant 2022

La Knowledge base



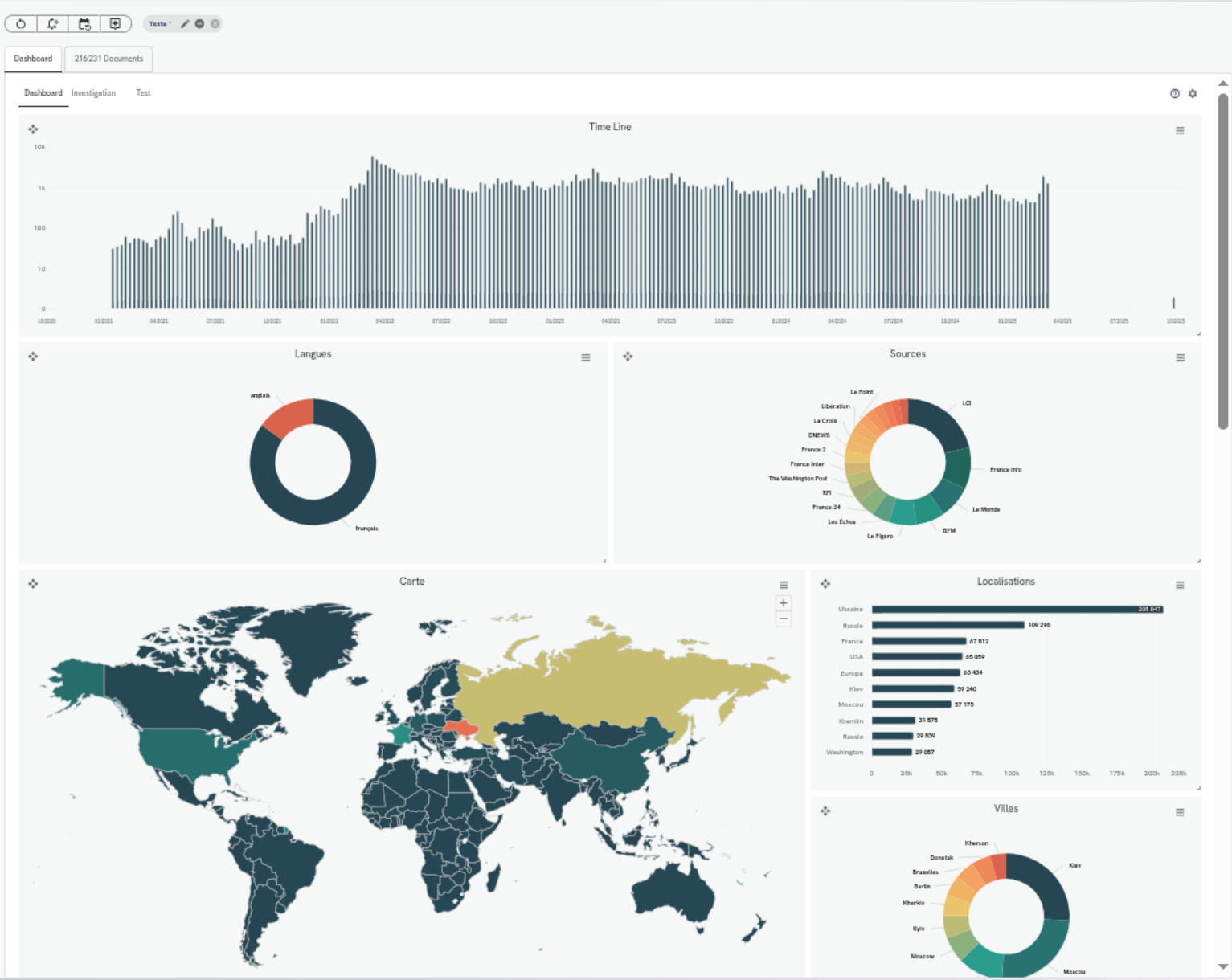
La Knowledge Base : simple et organisé.

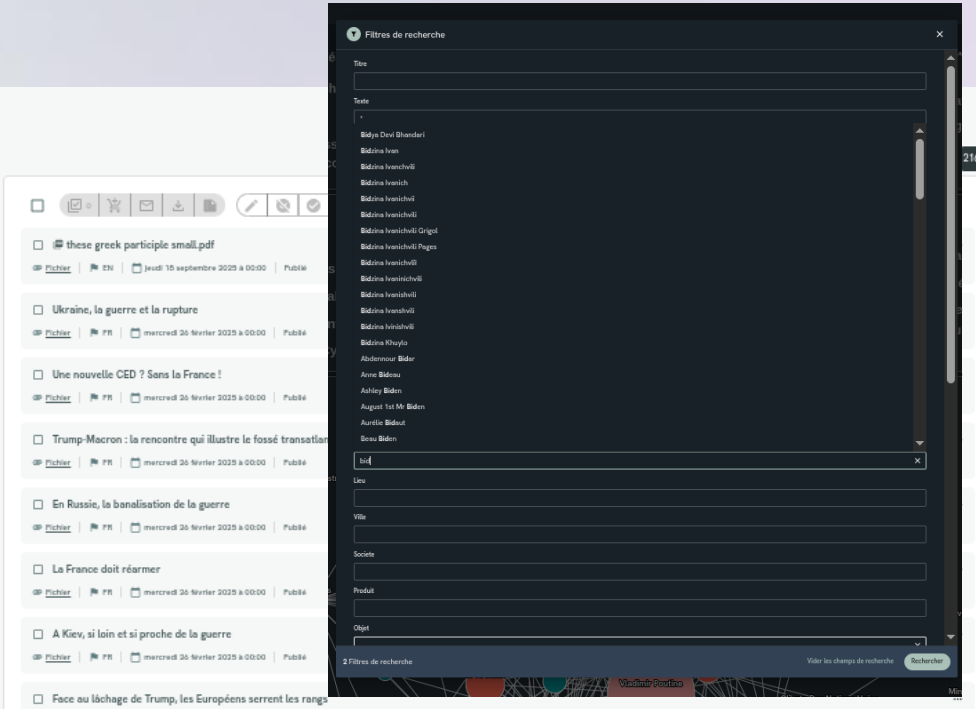


Le monde (infernal) des LLM : tout es ts possible, maut à quel prix ?!

LE BON TEMPS

© ChapsVision - All rights reserved





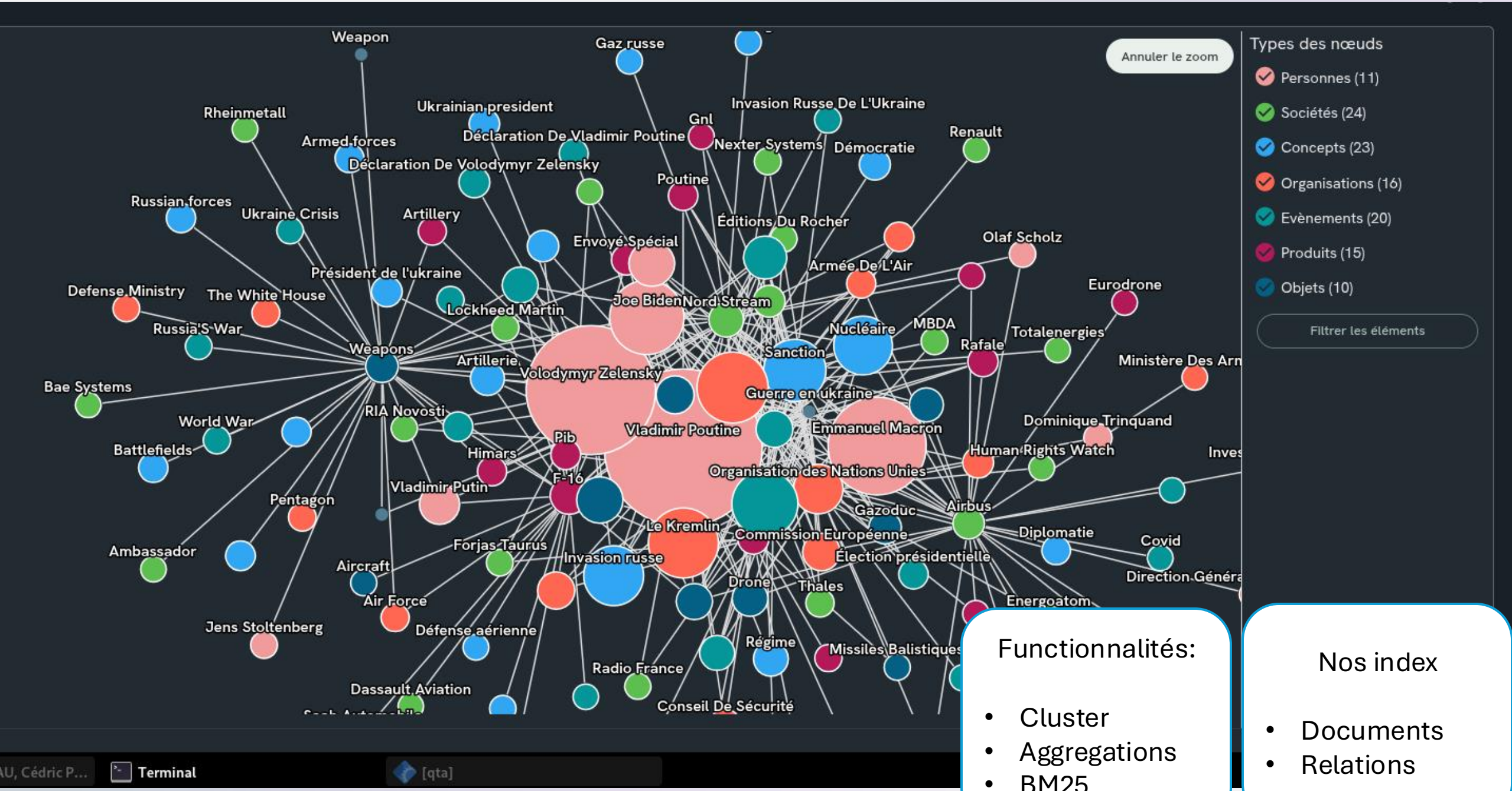
Fonctionnalités:

- Cluster
- Aggregations
- BM25

Nos index :

- Documents

© ChapsVision - All rights reserved



Functionnalités:

- Cluster
- Aggregations
- BM25

Nos index

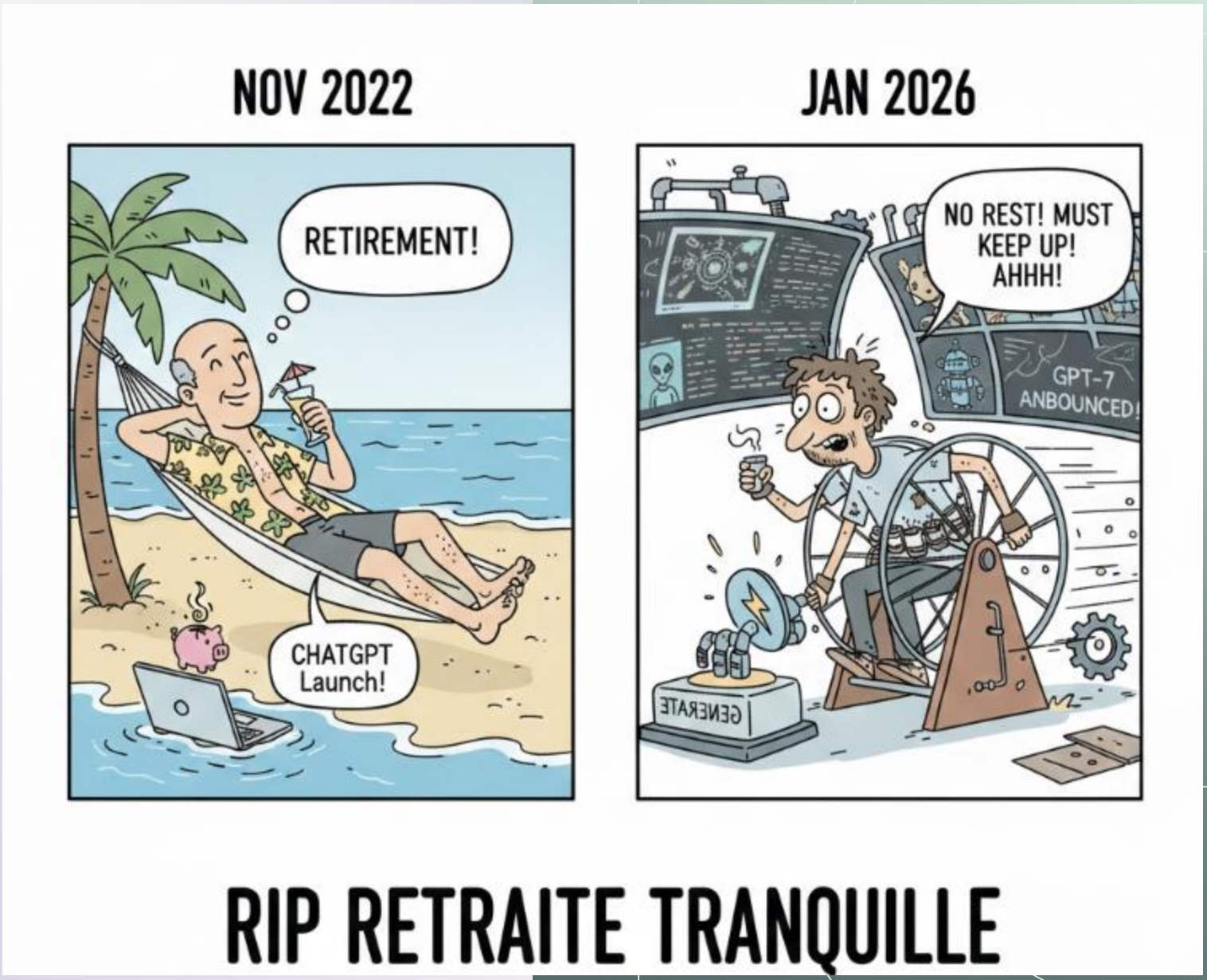
- Documents
- Relations

© ChapsVision - All rights reserved

0

Nov 2022

Date fatidique



© ChapsVision - All rights reserved

La roadmap agentique





RAG ?

perplexity



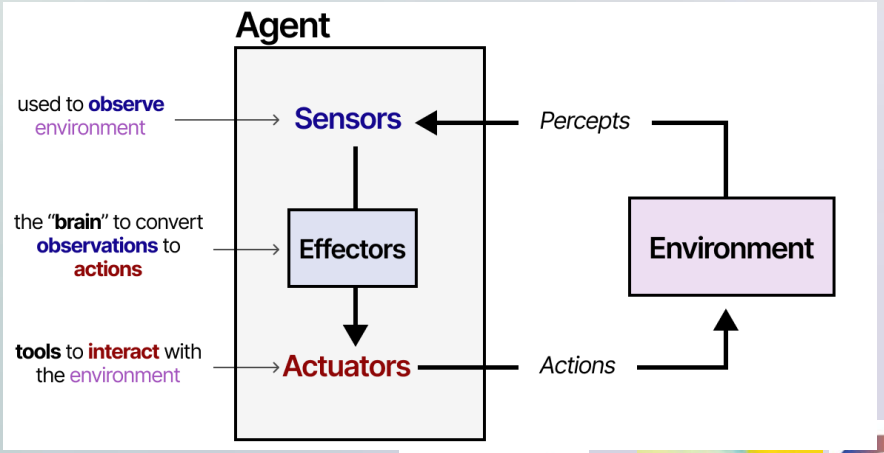
Qwen



Entreprise + Régalien



QWAM
by CHAPSVISION



ChatGPT

OpenAI o1

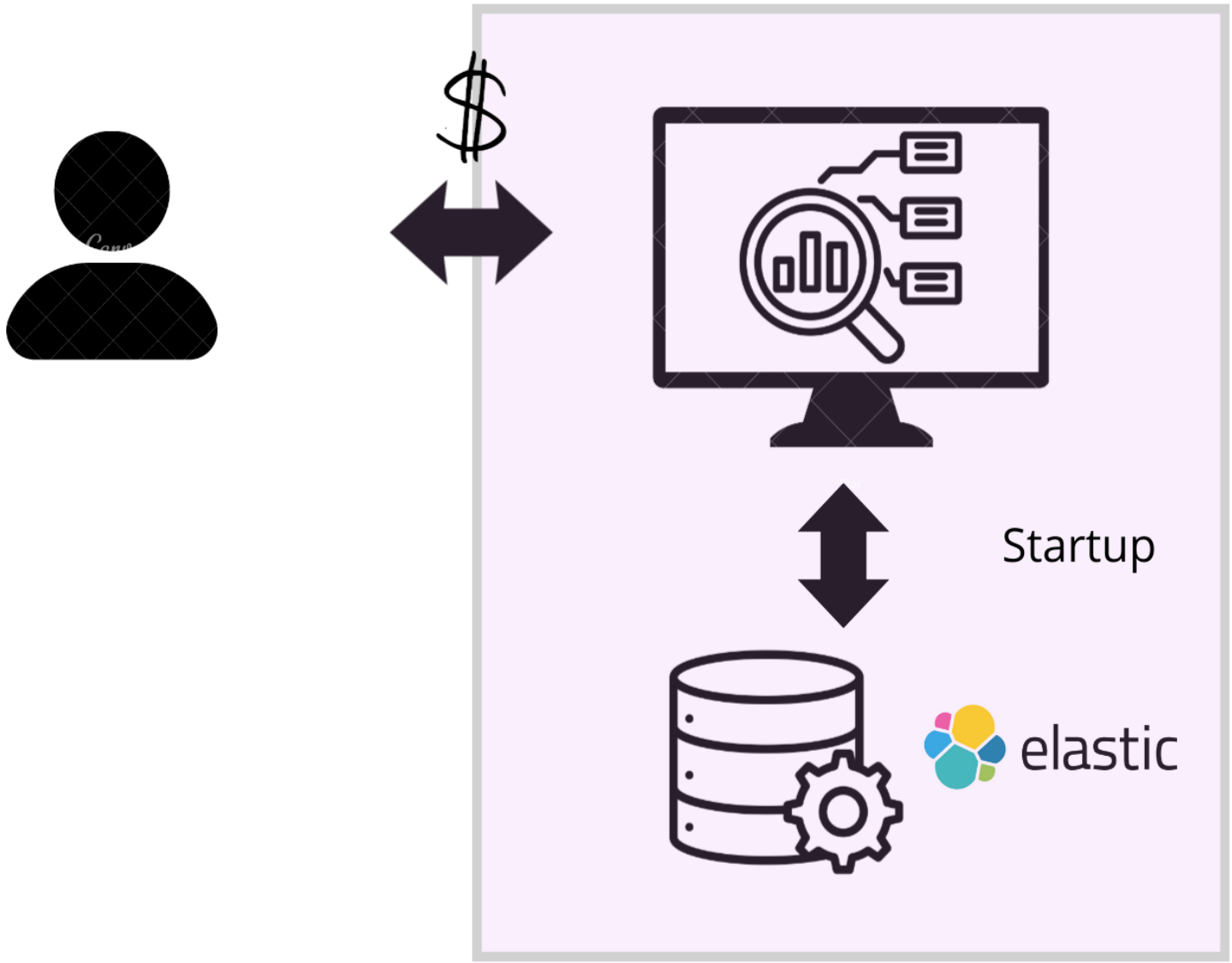
Agents orchestrés

Agents workflow

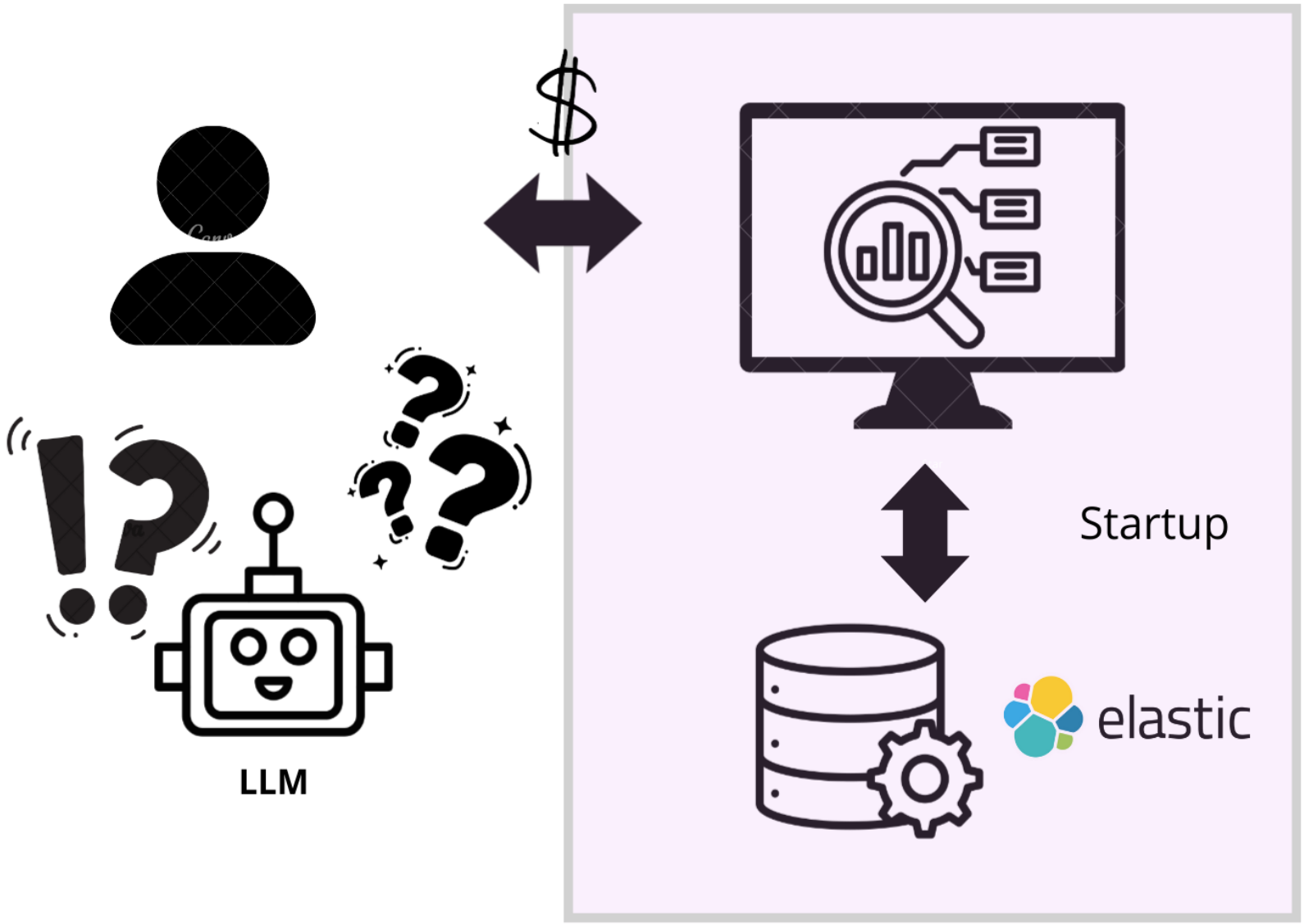
Agent conversationnel

© ChapsVision - All rights reserved

Chaps Vision



© ChapsVision - All rights reserved



© ChapsVision - All rights reserved

Pourquoi se lancer ?

→ Tout le monde dit : C'est très compliqué !

Effectivement: C'est trrrrrès compliqué. Peu de monde est prêt à maintenir l'effort dans la durée (c'est une course!)

→ Beaucoup en Cloud, peu On Premise

Certains clients sont régaliens. La scalabilité les LLM est le problème No1

→ Il y a beaucoup de frameworks

Oui, mais pas toujours ouverts. C'est une chaine que l'on doit maitriser totalement. Les frameworks rendent les upgrades difficiles. Les "rustines" sur l'esprit du framework finissent par fragiliser l'ensemble.

→ Nos clients sont partiellement équipés

Ils ont une application existante à laquelle ils veulent rajouter un RAG, un agent conversationnel, du NLP, etc ...

© ChapsVision - All rights reserved

01

RAG :

Connecter les LLMs au contenu





L'INTERFACE ULTIME

Le client de 2026 est (très) exigeant...

© ChapsVision - All rights reserved

La difficulté du contexte



Je vais au mammouth pour le dîner

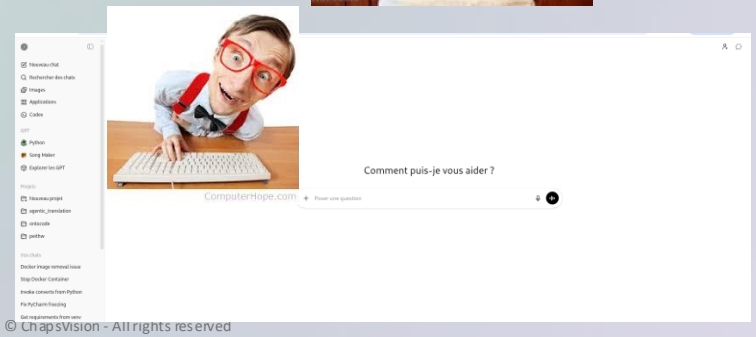






Je vais au Mammouth pour le dîner





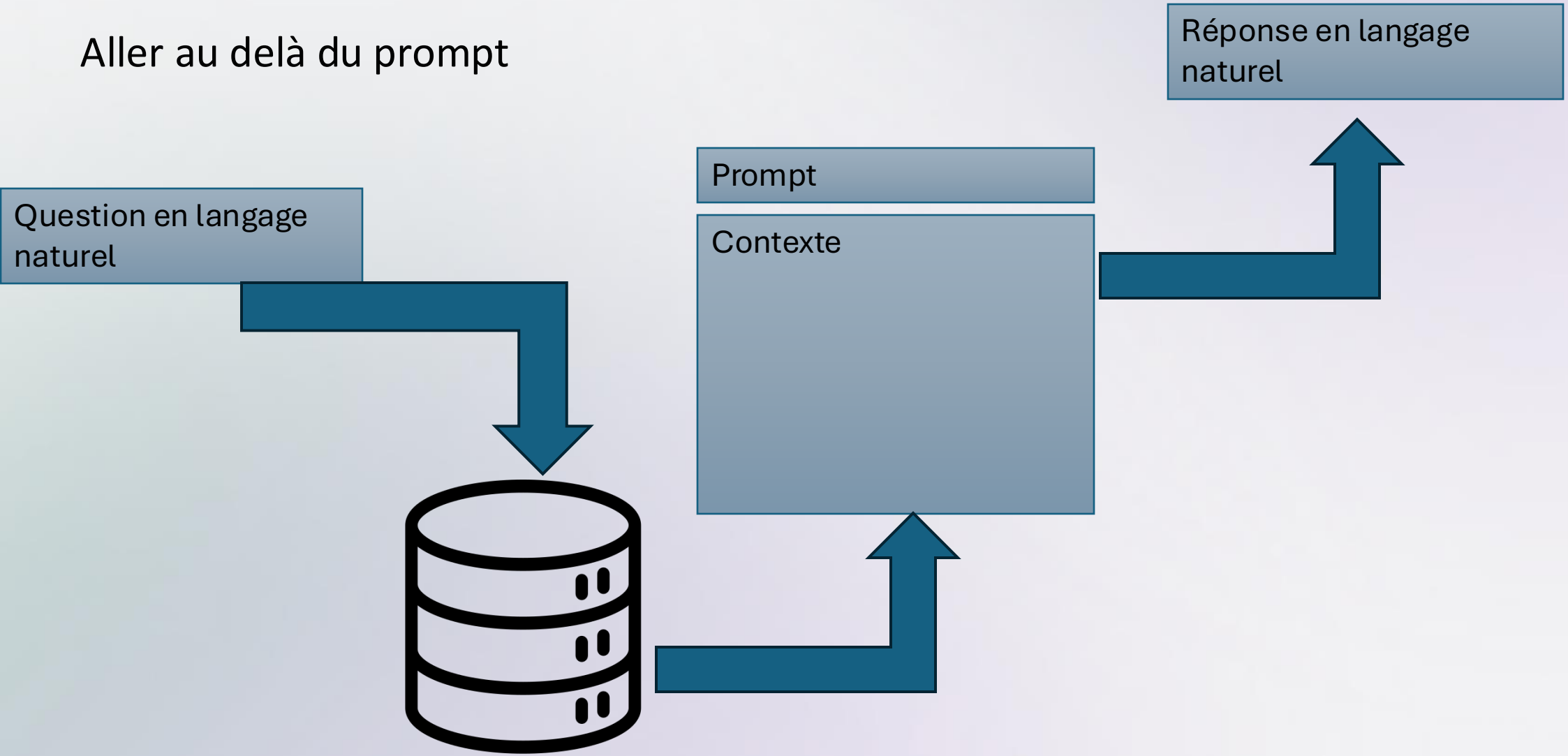
Dis moi tout sur le clonage des mammouths



© ChapsVision - All rights reserved

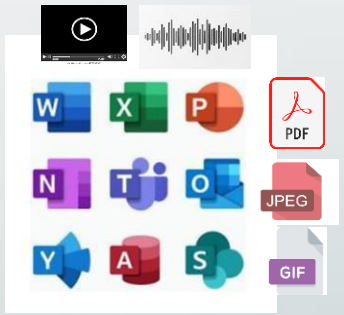
RAG : utiliser mes données dans le prompt

Aller au delà du prompt

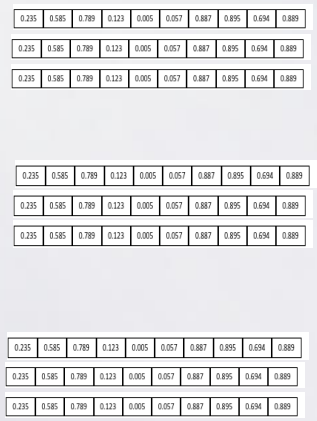


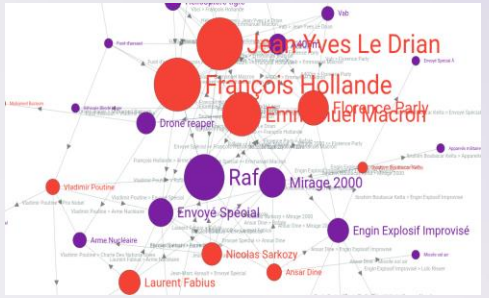
© ChapsVision - All rights reserved

Process des données











elasticsearch

Fichiers → Documents → Chunks → Embeddings → Relations → Stockage → Indexing

- OCR
- Traduction
- Description Caption
- Enrichissement
- Transcription

Fonctionnalités:

- Cluster
- BM25
- Dense_vector

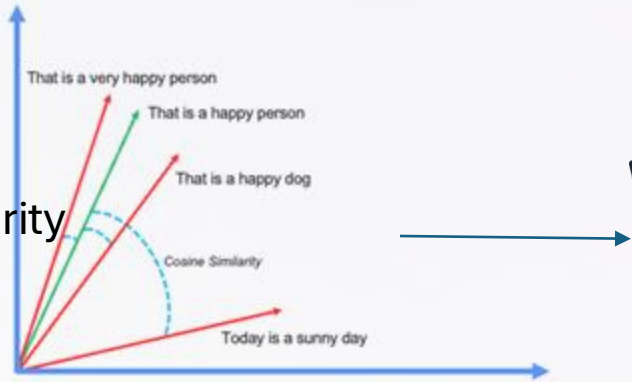
Nos index :

- Documents
- Relations
- Chunks

© ChapsVision - All rights reserved

Retrieval

Semantic search = embeddings cosine similarity





Who is president of the United states ?

Classic search = TF/IDF = BM25

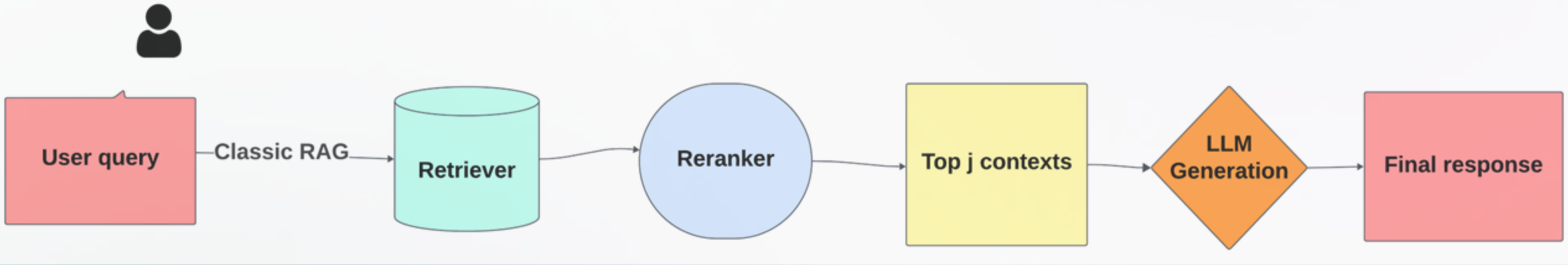
Who is **president** of the **United states** ?



Chunks → Embeddings → Espace latent

© ChapsVision - All rights reserved

RAG SIMPLE



Fonctionnalités:

- Cluster
- BM25
- Dense_vector
- KNN

Nos index :

- Documents
- Relations
- Chunks

© ChapsVision - All rights reserved

Le process du RAG

Quels sont les rapports entre le **président de la République** et les membres de la **junte militaire** au **Mali** ?

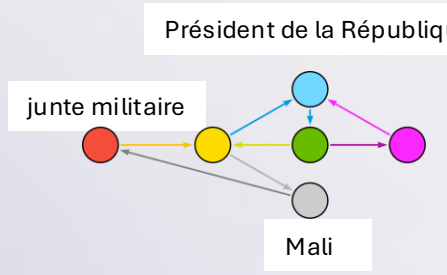
Routeur de stratégie

Question answering
Step back, reformulation, Parent document
TOT, COT, GOT

Expansion de requêtes

Qui est le président de la République ?
Qui sont les membres de la junte militaire Malienne ?
Quelles sont les relations entre la France et le Mali ?
Quels sont les conflits entre les gouvernements de France et du Mali ?

Traversée de graphes



Contextes



Reranker



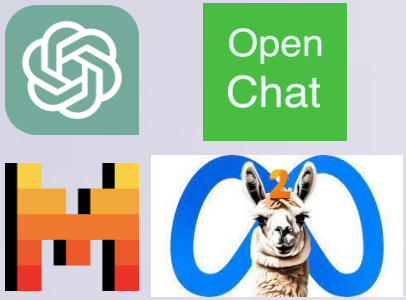
Prompt

You must answer the question...

+



LLM



Réponse

Les rapports entre le gouvernement français et la junte militaire au Mali se sont tendus pour plusieurs raisons :
1. Le président français a déclaré ....

© ChapsVision - All rights reserved

elastic





Ivan Monnier & Louis Jourdain

Technical architect NLP & AI engineer

CHAPSVISION





Traversée d'un Graphe, avec des millions de relations pour améliorer un RAG

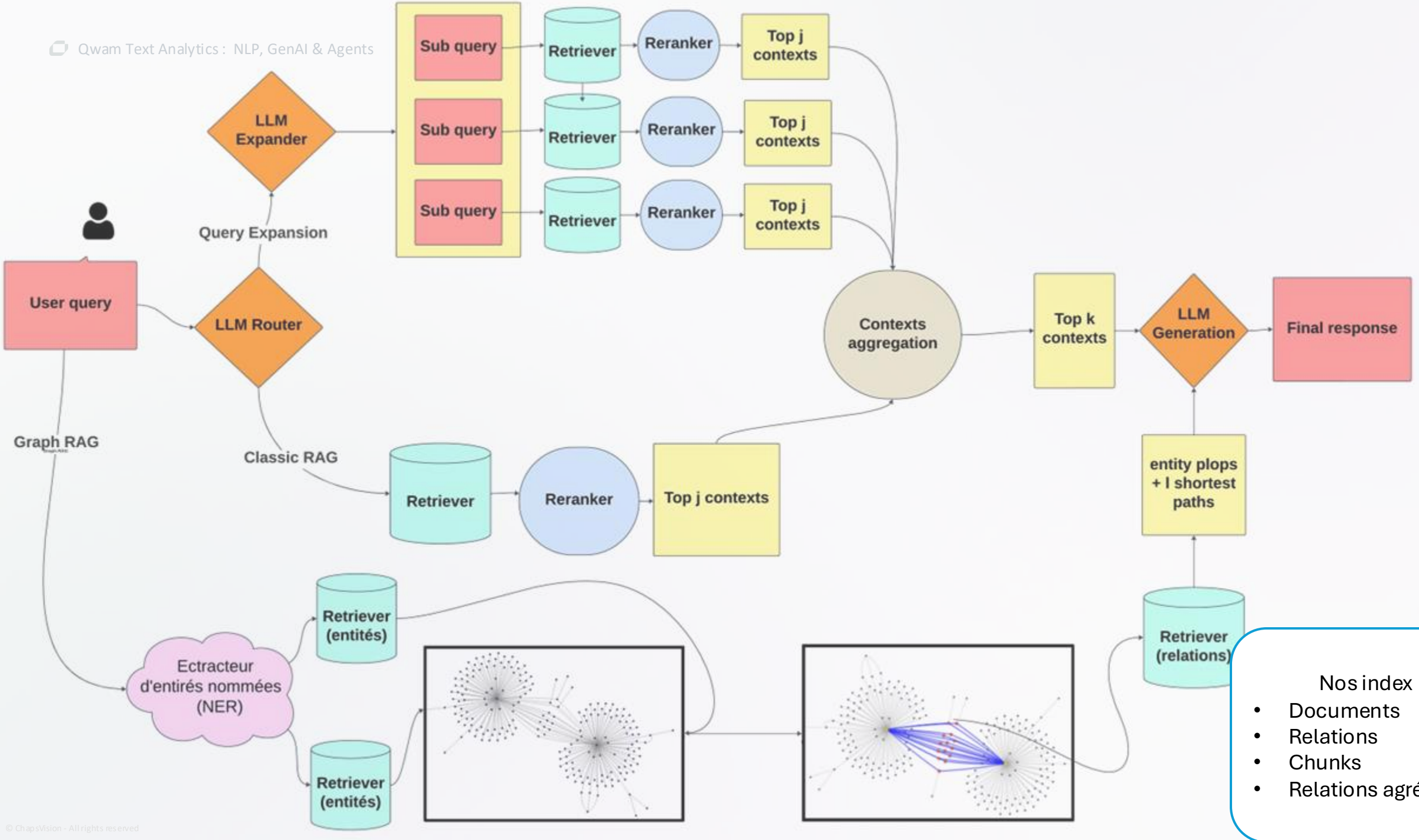
Sponsors

elastic

CANAL+

© ChapsVision - All rights reserved

Chaps Vision



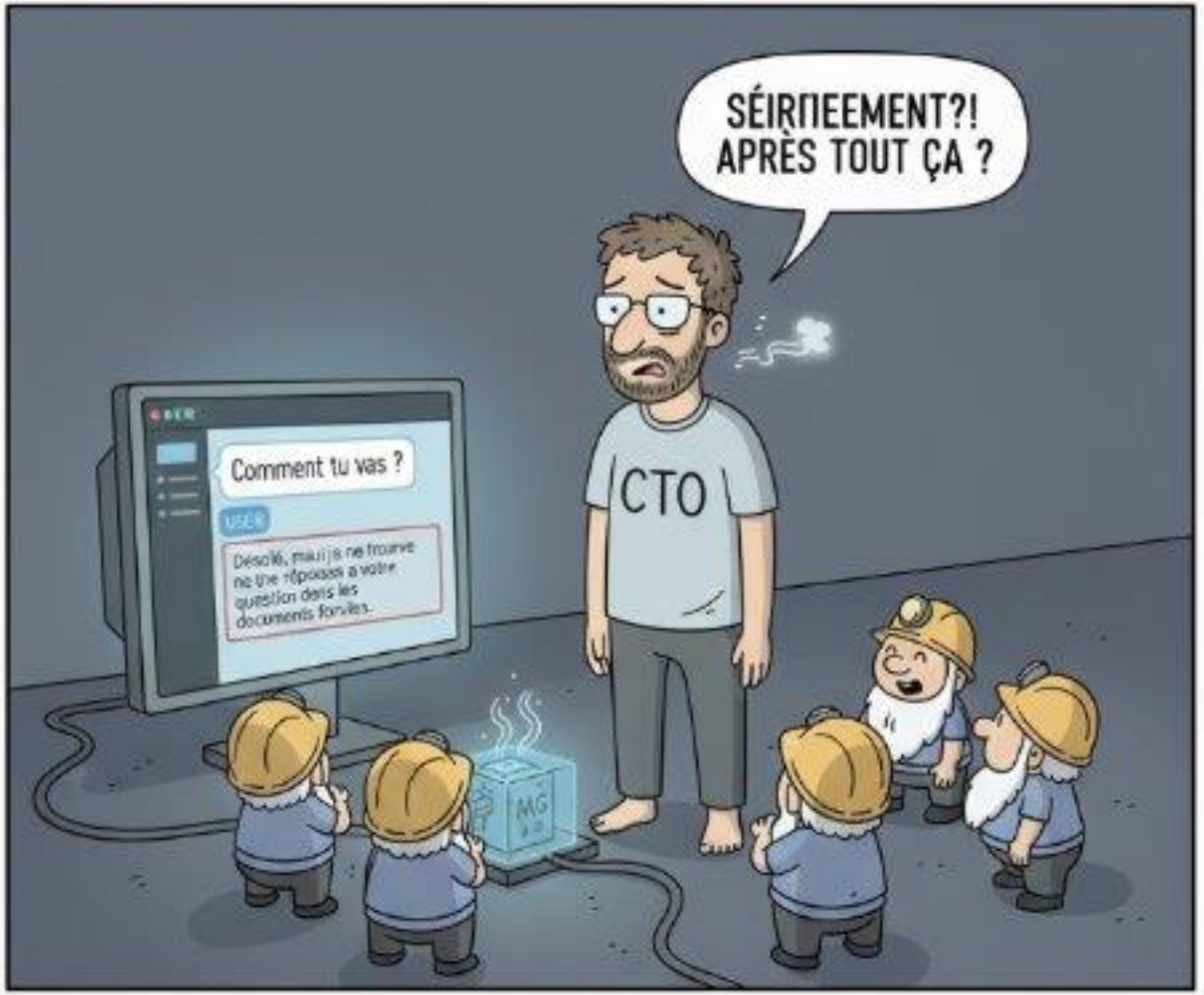
Nos index

- Documents
- Relations
- Chunks
- Relations agrégées

© ChapsVision - All rights reserved

02

Du RAG à l'Agent conversationnel



LE CTO ET LE RAG EXISTENTIEL

© ChapsVision - All rights reserved

Ce que sait faire un LLM

Reformulation

Résumé

Paraphrase

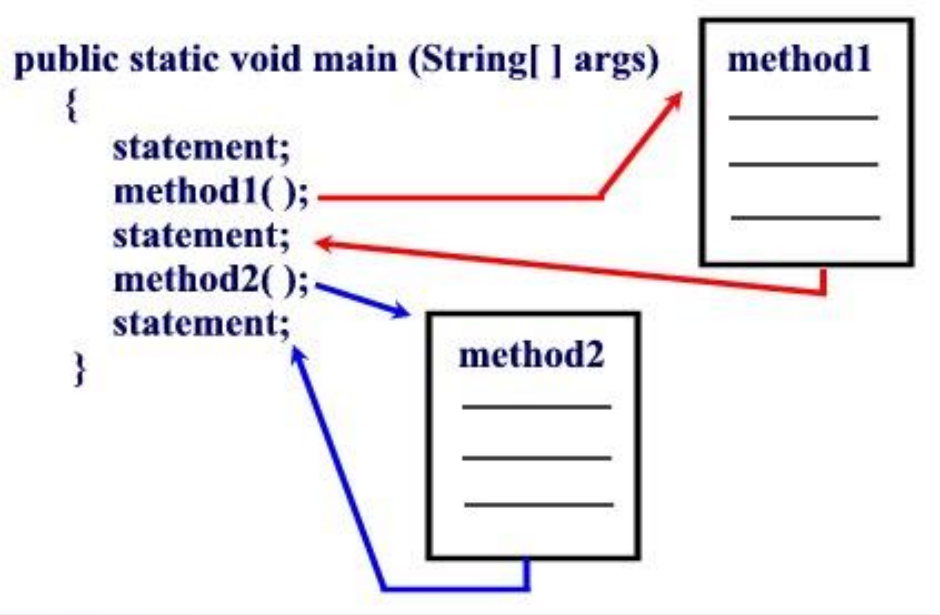
Traduction

G de RAG

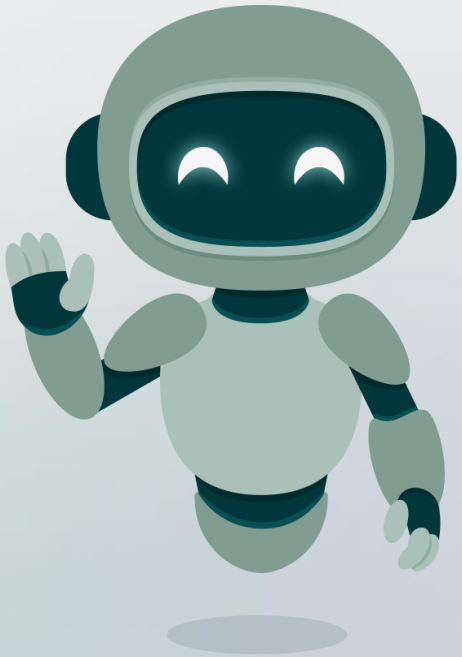
Function calling

Choisir dans une liste la fonction la plus adaptée selon sa description

Extraire certains paramètres du prompt



© ChapsVision - All rights reserved



The augmented LLM

LLM Agnosticism

Prompt Library

Tool Factory

Semantic Memory

© ChapsVision - All rights reserved

```python
tools = [{
    "type": "function",
    "function": {
        "name": "send_email",
        "description": "Send an email to a given recipient with a subject and message.
        "parameters": {
            "type": "object",
            "properties": {
                "to": {
                    "type": "string",
                    "description": "The recipient email address."
                },
                "subject": {
                    "type": "string",
                    "description": "Email subject line."
                },
                "body": {
                    "type": "string",
                    "description": "Body of the email message."
                }
            },
            "required": [
                "to",
                "subject",
                "body"
            ],
            "additionalProperties": False
        },
        "strict": True
    }
}]

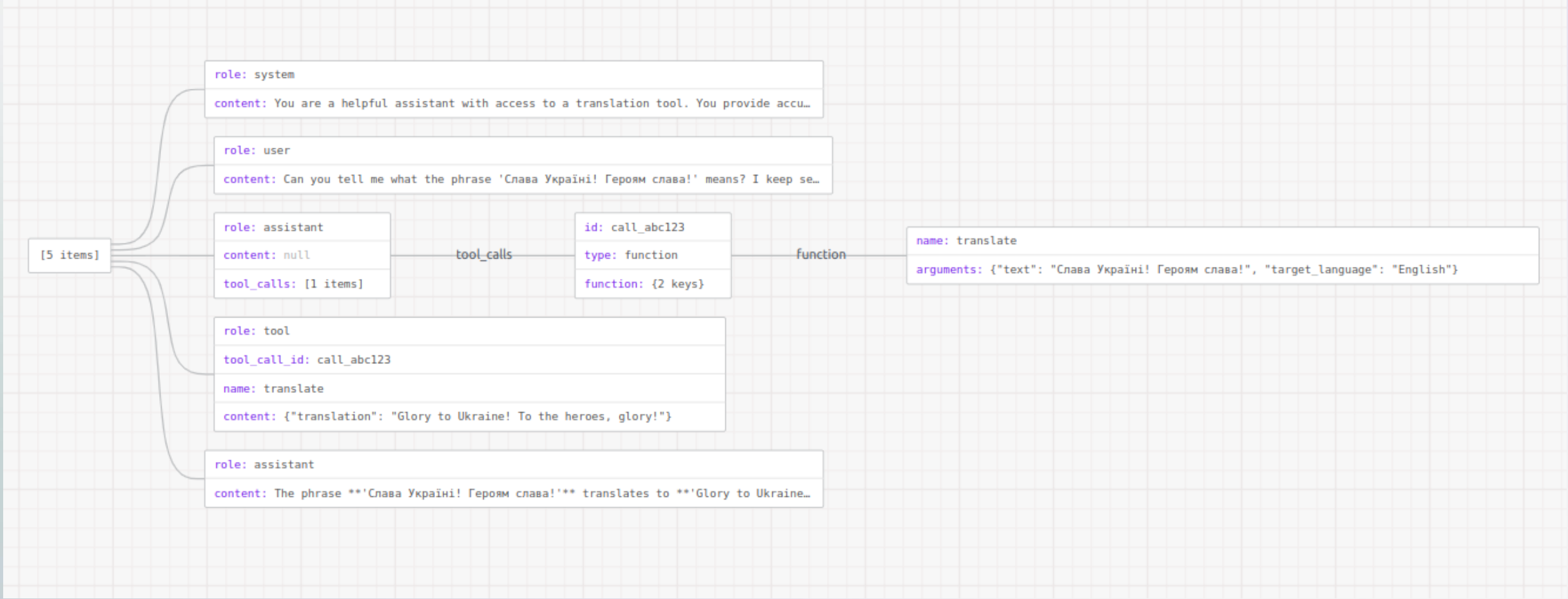
completion = client.chat.completions.create(
    model="gpt-4o",
    messages=[{"role": "user", "content": "Can you send an email to ilan@example.com a
    tools=tools
)
```

```
<OpenAIObject at 0x109cf5f90> JSON: {
  "role": "assistant",
  "content": null,
  "function_call": {
    "name": "get_current_weather",
    "arguments": "{\n  \"location\": \"Boston, MA\"\n}"
  }
}
```

© ChapsVision - All rights reserved

Chaps Vision

Un format standardisé comme JSON



© ChapsVision - All rights reserved

02

Implémenter la mémoire



© ChapsVision - All rights reserved

Les différents types de mémoire implémentés dans LangChain / LangGraph

Memory Type	Description
ConversationBufferMemory	Stores the entire conversation history in memory without any additional processing. This allows the model to access all previous interactions, ensuring contextually relevant responses. However, it may lead to high memory usage as the conversation grows. PYTHON.LANGCHAIN.COM
ConversationBufferWindowMemory	Maintains only the most recent interactions up to a specified number (`k`). This approach helps in managing memory usage by limiting the context to recent exchanges, which is beneficial when the full history isn't necessary for understanding the current interaction. AIGUIDO.COM
ConversationSummaryMemory	Summarizes the conversation history and provides this summary as context for future interactions. This method helps in keeping the context concise, especially useful in long conversations where maintaining the entire history isn't practical. AIGUIDO.COM
ConversationSummaryBufferMemory	Combines summarization with a buffer of recent interactions. It maintains a moving summary of the conversation and appends recent exchanges until a token limit is reached, after which older interactions are summarized. This hybrid approach balances between detailed recent context and summarized past interactions. LANGCHAIN.READTHEDOCS.IO
ConversationTokenBufferMemory	Keeps a buffer of recent interactions based on token count rather than the number of interactions. Once the token limit is exceeded, it removes the oldest messages to stay within the limit. This method ensures that the memory usage is controlled based on the token count, which is crucial for models with token limitations. LANGCHAIN.READTHEDOCS.IO
Entity Memory	Extracts and stores information about specific entities mentioned during the conversation. This allows the model to remember and refer back to these entities in future interactions, enhancing the coherence and relevance of the conversation. TOWARDSAI.NET
Conversation Knowledge Graph Memory	Integrates with an external knowledge graph to store and retrieve information about knowledge triples discussed in the conversation. This structured approach enables the model to understand and utilize relationships between entities, providing more informed responses. TOWARDSAI.NET
VectorStore-Backed Memory	Utilizes a vector store to store and retrieve information based on semantic similarity. This method allows the model to access relevant past interactions or documents that are semantically similar to the current context, improving the relevance of responses. TOWARDSAI.NET

© ChapsVision - All rights reserved

Cognitive Architectures for Language Agents

Theodore R. Sumers* **Shunyu Yao*** **Karthik Narasimhan** **Thomas L. Griffiths**
Princeton University
{sumers, shunyuy, karthikn, tomg}@princeton.edu

Reviewed on OpenReview: *https://openreview.net/forum?id=1i6ZCvflQJ*

[cs.AI] 15 Mar 2024

Abstract

Recent efforts have augmented large language models (LLMs) with external resources (e.g., the Internet) or internal control flows (e.g., prompt chaining) for tasks requiring grounding or reasoning, leading to a new class of *language agents*. While these agents have achieved substantial empirical success, we lack a framework to organize existing agents and plan future developments. In this paper, we draw on the rich history of cognitive science and symbolic artificial intelligence to propose Cognitive Architectures for Language Agents (CoALA). CoALA describes a language agent with modular memory components, a structured action space to interact with internal memory and external environments, and a generalized decision-making process to choose actions. We use CoALA to *retrospectively* survey and organize a large body of recent work, and *prospectively* identify actionable directions towards more capable agents. Taken together, CoALA contextualizes today's language agents within the broader history of AI and outlines a path towards language-based general intelligence.

© ChapsVision - All rights reserved

Memory. Building on psychological theories, Soar uses several types of memory to track the agent's state (Atkinson and Shiffrin, 1968). *Working memory* (Baddeley and Hitch, 1974) reflects the agent's current circumstances: it stores the agent's recent perceptual input, goals, and results from intermediate, internal reasoning. *Long term memory* is divided into three distinct types. *Procedural* memory stores the production system itself: the set of rules that can be applied to working memory to determine the agent's behavior. *Semantic* memory stores facts about the world (Lindes and Laird, 2016), while *episodic* memory stores sequences of the agent's past behaviors (Nuxoll and Laird, 2007).

Memory Type	What is Stored	Human Example	Agent Example
Semantic	Facts	Things I learned in school	Facts about a user
Episodic	Experiences	Things I did	Past agent actions
Procedural	Instructions	Instincts or motor skills	Agent system prompt

© ChapsVision - All rights reserved

Memory Innovation Timeline

1. 2019-2021: Early retrieval, scratchpads
2. 2022: MemPrompt (persistent key-value)
3. Early 2023: Generative Agents (reflection)
4. Mid-Late 2023: CoALA, MemGPT, MemoryBank
5. 2024: LoCoMo benchmark (300+ turns)
6. 2025: A-MEM, LongMemEval, Mem0, Memory-R1

Memory in the Age of AI Agents: A Survey

Forms, Functions and Dynamics

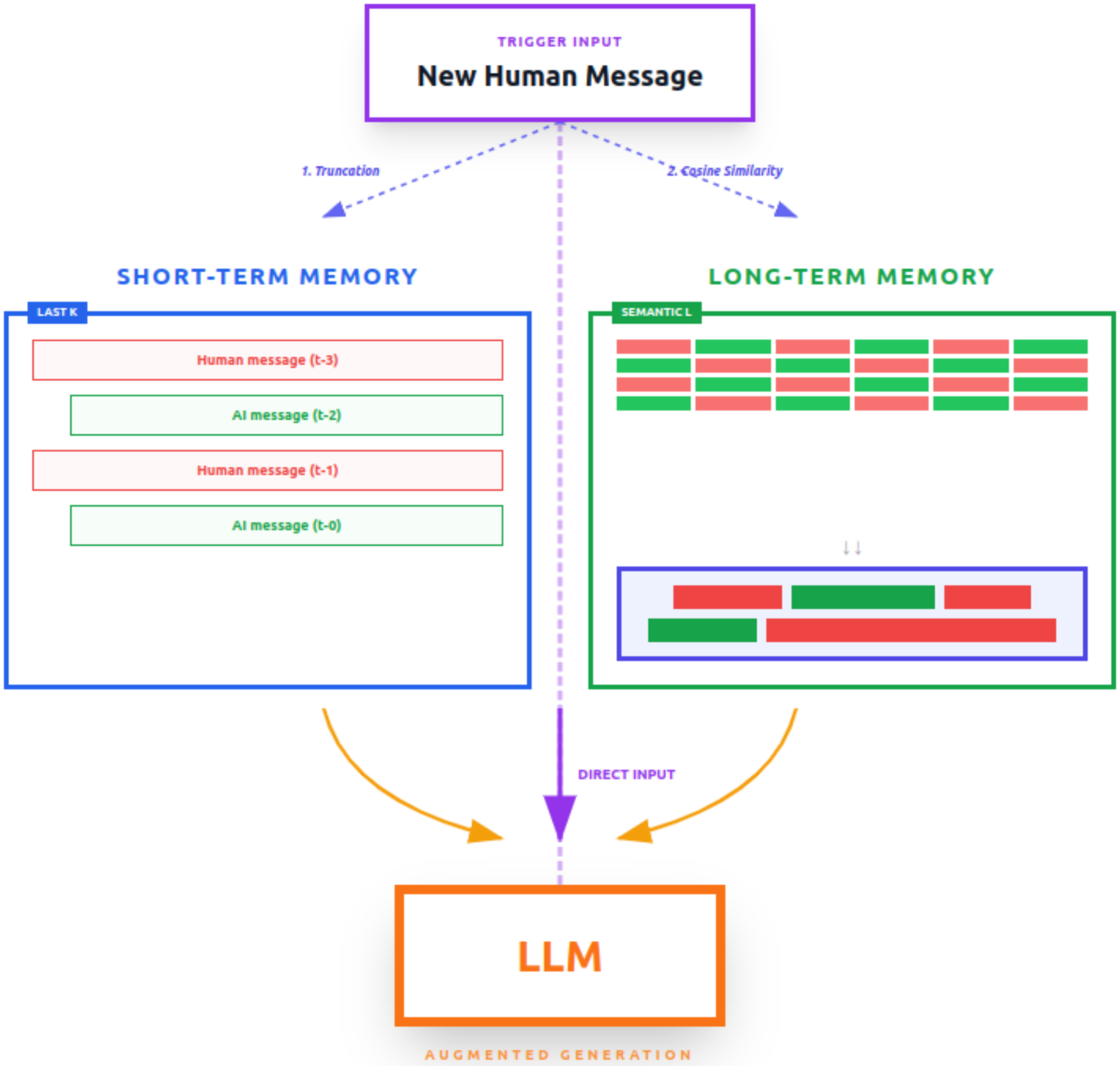
Yuyang Hu[†], **Shichun Liu**[†], **Yanwei Yue**[†], **Guibin Zhang**[†], **Boyang Liu**, **Fangyi Zhu**, **Jiahang Lin**, **Honglin Guo**, **Shihan Dou**, **Zhiheng Xi**, **Senjie Jin**, **Jiejun Tan**, **Yanbin Yin**, **Jiongnan Liu**, **Zeyu Zhang**, **Zhongxiang Sun**, **Yutao Zhu**, **Hao Sun**, **Boci Peng**, **Zhenrong Cheng**, **Xuanbo Fan**, **Jiaxin Guo**, **Xinlei Yu**, **Zhenhong Zhou**, **Zewen Hu**, **Jiahao Huo**, **Junhao Wang**, **Yuwei Niu**, **Yu Wang**, **Zhenfei Yin**, **Xiaobin Hu**, **Yue Liao**, **Qiankun Li**, **Kun Wang**, **Wangchunshu Zhou**, **Yixin Liu**, **Dawei Cheng**, **Qi Zhang**, **Tao Gui**[‡], **Shirui Pan**, **Yan Zhang**[‡], **Philip Torr**, **Zhicheng Dou**[‡], **Ji-Rong Wen**, **Xuanjing Huang**[‡], **Yu-Gang Jiang**, **Shuicheng Yan**[‡]

[†]Core Contributors with Names Listed Alphabetically. Project Organizer. [‡]Core Supervisors.

Affiliations: National University of Singapore, Renmin University of China, Fudan University, Peking University, Nanyang Technological University, Tongji University, University of California San Diego, Hong Kong University of Science and Technology (Guangzhou), Griffith University, Georgia Institute of Technology, OPPO, Oxford University

© ChapsVision - All rights reserved

Chaps Vision



Challenge :

- S'adapter dynamiquement à la taille de la fenêtre de contexte du LLM utilisé
- Se souvenir de l'enchaînement logique des messages
- Être capable de réexploiter des éléments pertinents mentionnés plus tôt dans la conversation s'ils sont utiles pour mieux répondre au tour suivant
- Parfois, un tour de conversation (surtout une réponse de LLM verbeux...) sont trop gros pour être récupérés par recherche vectorielle... Il faut les découper !

Chunking des conversations

```python
class ConversationalFragment:
    """
    Represents a single fragment of a conversation.

    A conversational turn may be split into multiple frag
    and each fragment is tracked with a unique fragment_i
    """
    def __init__(
        self,
        user_id: str,
        role: str,
        content: str,
        conversation_id: str,
        conversational_turn_id: int,
        conversation_run_id: str,
        llm: Optional[str] = None,
        conversation_title: Optional[str] = None,
        fragment_id: Optional[int] = None,
        content_vector: Optional[List[float]] = None,
        timestamp: Optional[datetime] = None,
        tool_calls: Optional[List] = None,
        tool_name: Optional[str] = None,
        tool_call_id: Optional[str] = None,
        artifact: Optional[Dict[str, Any]] = None,
        reasoning_content: Optional[str] = None
    ):
        """
        Initialize a new conversational fragment.
```

```python
class Conversation:
    """
    Represents metadata and content pointers for a full conversation.
    Tracks fragments and turns associated with a user session.
    """

    def __init__(
        self,
        conversation_id: str,
        user_id: str,
        conversation_run_id: Optional[str] = None,
        conversation_title: Optional[str] = None,
        llm: Optional[str] = None,
        fragments: List[ConversationalFragment] = None,
        topics=None,
        system_prompt=None
    ):
        """
        Initialize a Conversation container.

        Args:
            conversation_id (str): Unique identifier for the conversation.
            user_id (str): User associated with this conversation.
            fragments (List[ConversationalFragment], optional): Initial list of fragments.
        """
        self.conversation_id = conversation_id
        self.conversation_run_id = conversation_run_id
        self.conversation_title = conversation_title
        self.llm = llm
        self.user_id = user_id
        self.fragments = fragments if fragments is not None else []
        self.fragment_count = len(self.fragments)
        self.turn_count = len(set(f.conversational_turn_id for f in self.fragment
        self.last_updated = self._compute_last_updated()
        self.topics = topics
        self.system_prompt = system_prompt
        self.current_token_usage = {
            "input_tokens_count": 0,
            "output_tokens_count": 0,
            "total_tokens_count": 0
        }
```

Nos index

- Documents
- Relations
- Chunks
- Relations agrégées
- Assistant

© ChapsVision - All rights reserved

Search AI in 2026 is Agent Memory

Han Xiao

VP of AI, Elastic
Formerly Founder & CEO, Jina AI

April 2026

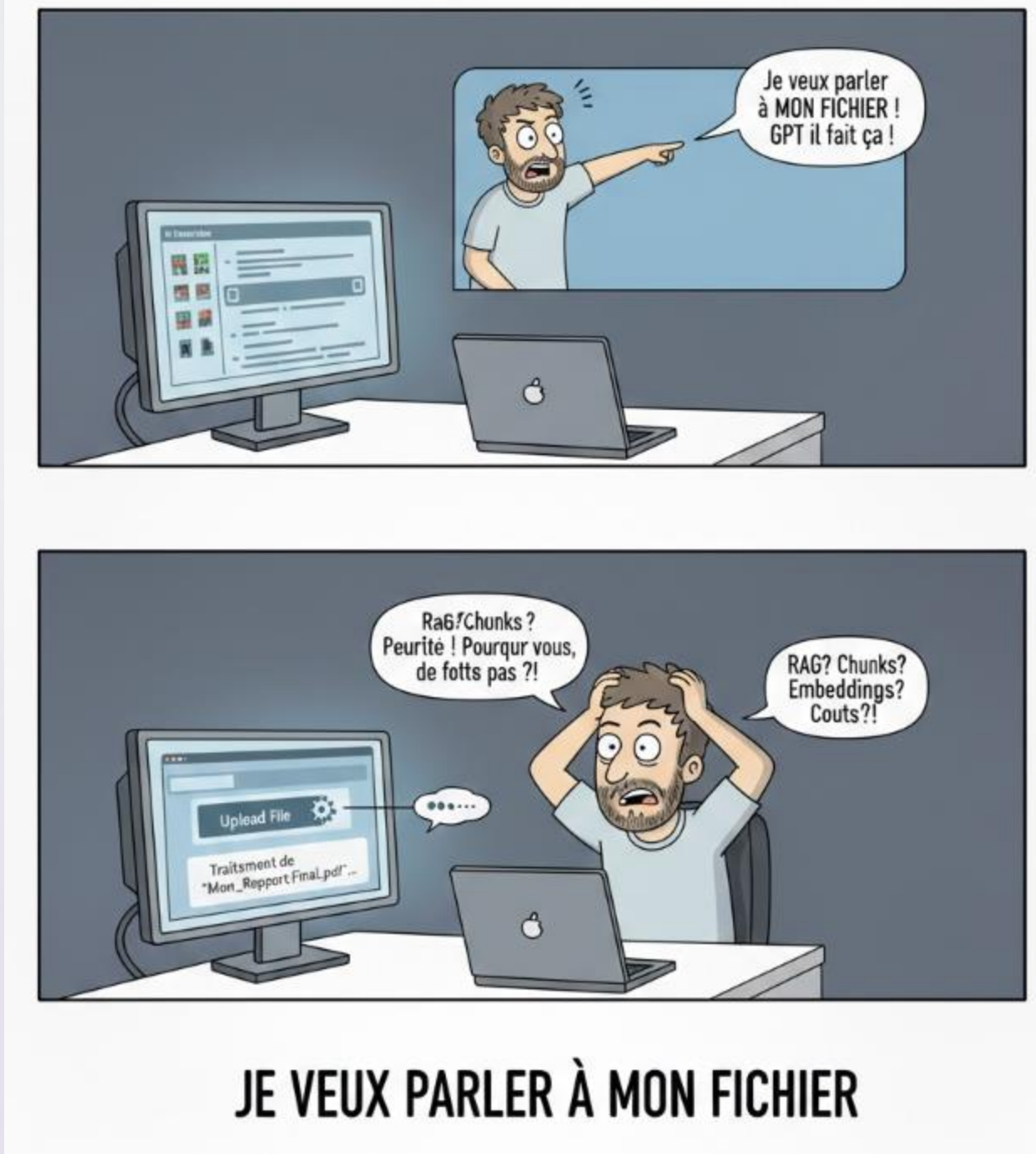
Technique	Adopters	Verdict
Vector semantic retrieval	Mem0, Supermemory, Cognee, HydraDB, Zep, MemOS	**Strong consensus**
LLM fact/entity extraction	Mem0, Zep, Supermemory, Cognee, HydraDB, Hindsight, OpenViking, MemOS	**Strong consensus**
Graph + vector hybrid	Mem0, Zep, Supermemory, Cognee, HydraDB, Hindsight, MemOS	**Strong consensus**
Memory atomization	Mem0, Supermemory, Cognee, HydraDB	**Strong consensus**
Knowledge conflict resolution	Mem0, Zep, Supermemory, HydraDB, Hindsight	**Strong consensus**
Temporal awareness (bi-temporal)	Zep, Supermemory, HydraDB, Hindsight	Medium-strong
BM25 + dense hybrid search	Zep, HydraDB	Medium
Memory lifecycle mgmt (OS paradigm)	MemOS, Letta	Emerging
LLM self-managed memory	Letta only	Academic, high prod risk
Pure SQL (no vector/graph)	Memori only	Contrarian
Git-style append-only graph	HydraDB only	Genuine innovation
Biologically-inspired decay	HydraDB, Supermemory	Experimental
Post-deploy graph optimization	Cognee only	Promising, no data
File-system context management	OpenViking only	New paradigm, unproven

© Chaps

Chaps Vision

04

Laissez-moi parler à MON fichier !



© ChapsVision - All rights reserved

Hi there ! I want to duplicate the Canvas functionnality in my AI assistant. What do you think ?

Hey 👋 That's a pretty bold move! Duplicating something like Canvas inside your AI assistant makes a lot of sense if you want to support **structured, persistent, multi-turn content creation** (documents, plans, code, slides, etc.) rather than just ephemeral chat.

Here's how I'd break it down:

🔑 Why it's valuable

- **Memory anchor**: Canvas acts as a "living document" tied to a conversation, which makes the assistant less ephemeral and more collaborative.
- **Iteration space**: Instead of users copying chat outputs into Word/Docs/Notion, you give them a place to refine drafts inside the assistant.
- **Artifact management**: If you already have an Artifact Vault, Canvas-like objects could just be one special artifact type (editable + versioned).

🧩 Core features to replicate

1. **Document object model**
   - Each canvas = persistent artifact linked to a conversation or project.
   - Rich text format: Markdown or JSON schema (to handle sections, headings, blocks).
   - Versioning built-in.
2. **Operations / tools**
   - `create_canvas`
   - `get_canvas_content`
   - `update_canvas` (replace whole text, or diff-patch updates)
   - `comment_canvas` (LLM suggestions without overwriting text)
3. **Integration with chat**
   - LLM can write *into* the canvas instead of just replying in chat.
   - Chat messages can be linked to specific parts of the canvas (anchored edits).
   - Assistant can "propose changes" → user approves → document updates.
4. **UI layer**
   - In chat view: small preview + "open full canvas" button.

© ChapsVision - All rights reserved

🗂️ The Artifact Vault File System

Goal:

Provide a secure, modular workspace for all AI-generated or user-uploaded materials — from raw files to collaborative documents (Qwanvas).

📁 Global Structure

bash Copier le code

```
/usr/bases/{project_name}/
 └── artifacts/
      ├── files/                        ← physical artifacts (documents, images…)
      │    └── {user_id}/
      │         └── {conversation_id}/
      │              ├── vault.json          ← metadata index
      │              ├── conversation_artifact1.png
      │              ├── conversation_artifact2.txt
      │              └── ...
      │
      └── qwanvas/                      ← structured, editable canvases
           └── {user_id}/
                └── {conversation_id}/
                     ├── plan.md
                     ├── content.md
                     └── history/
                          ├── version_0.md
                          ├── version_1.md
                          └── ...
```

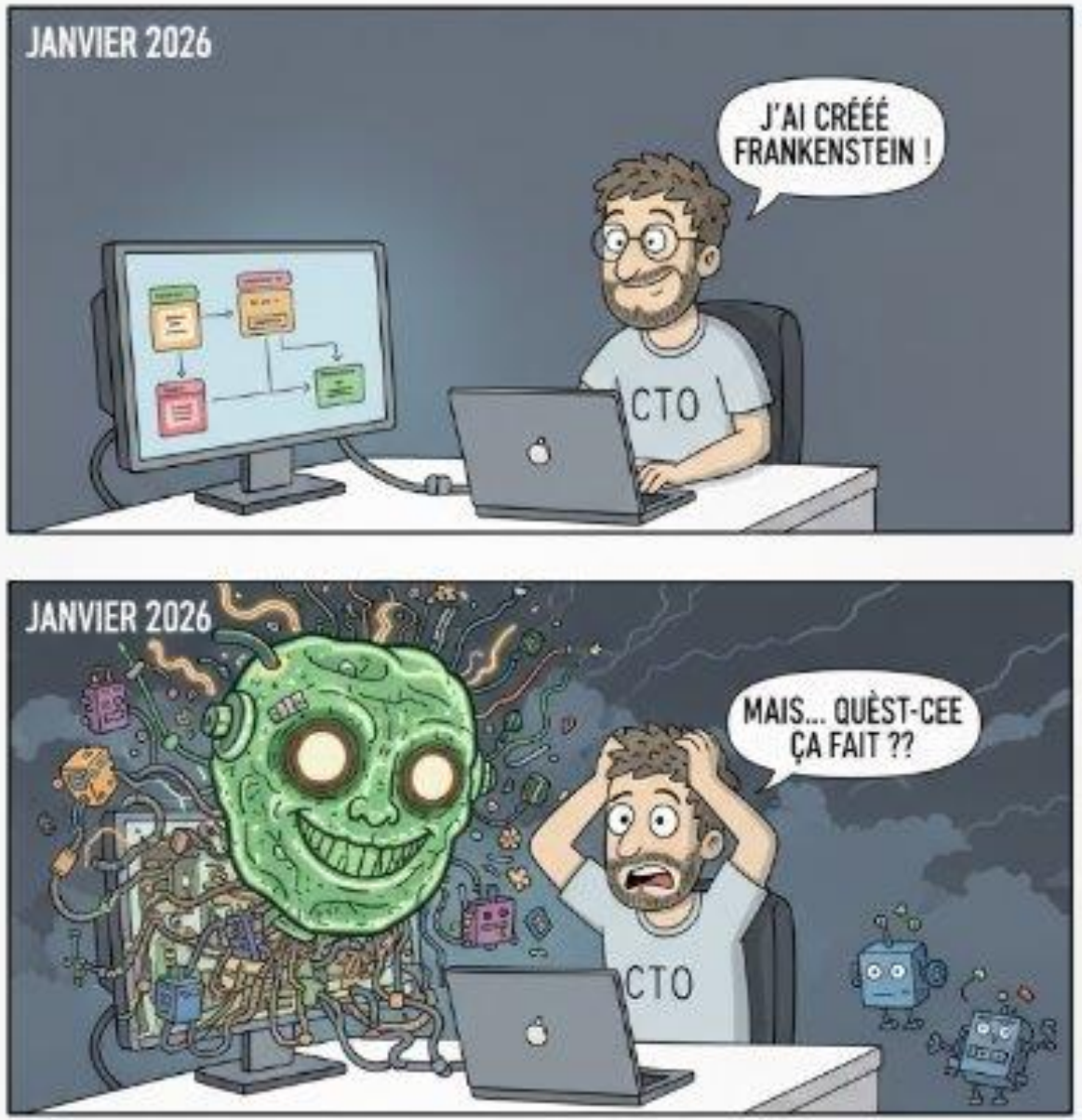
Nos index

- Documents
- Relations
- Chunks
- Relations agrégées
- Assistant
- Artifacts
- Artifacts_chunks

© ChapsVision - All rights reserved

05

Agentic Hype



© ChapsVision - All rights reserved

Les niveaux agentiques

Le RAG

- One shot
- Un seul outil

L'agent conversationnel

- Conversation
- **LLM augmenté** ..
- … avec plusieurs outils

Le workflow

- Super outils =
- Assemblage Outils + LLM
- Logique pré déterminée

L'agent orchestrateur

- Plusieurs **LLM augmentés**…
- … Chacun une tâche….
- … logique gérée par l'agent orchestrateur …
- … itération jusqu'au résultat évalué



© ChapsVision - All rights reserved

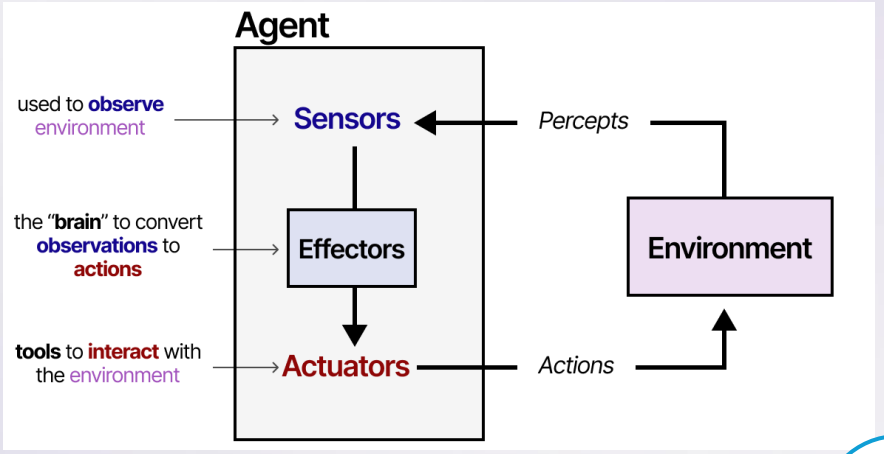
Agentique: effectuer des tâches complexes

Limitation: Un tool = 1 tâche = 1 agent

On programme tout à l'avance



On laisse le système décider



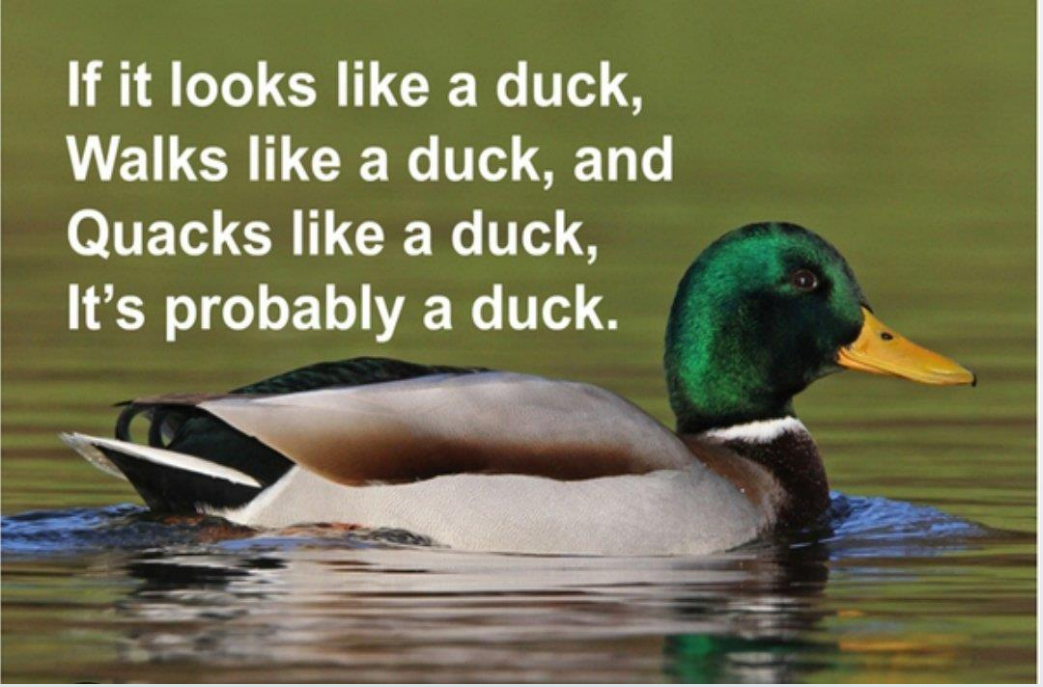
Fonctionnalités:

- Cluster
- BM25
- Dense_vector
- KNN
- Filtered search
- BBQ

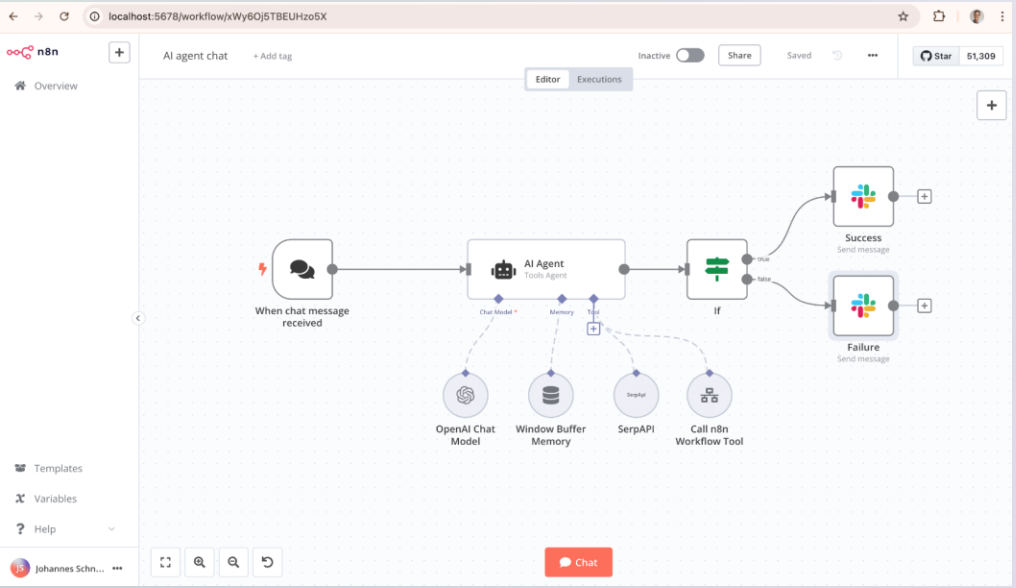
Indexes so far:

- Documents
- Relations
- Chunks
- Base de graphe
- Conversations
- Conversations chunks
- Artifacts
- Observability nodes
- Observability runs

© ChapsVision - All rights reserved

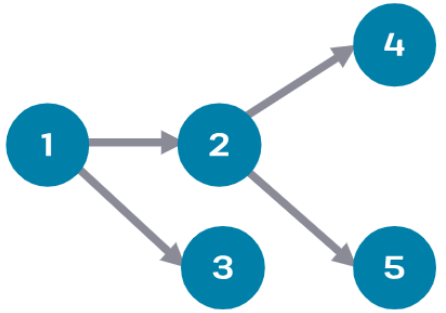


What does it look like ?



Directed Acyclic Graph (DAG)

A **directed acyclic graph (DAG)** is a conceptual representation of a series of activities. The order of the activities is depicted by a graph, which is visually presented as a set of circles. Each circle represents an activity, some of which are connected by lines, representing the flow from one activity to another. Each circle is known as a "vertex," and each line is known as an "edge." "Directed" means that each edge has a defined direction. Each edge necessarily represents a single directional flow from one vertex to another. "Acyclic" means that there are no loops (i.e., "cycles") in the graph. For any given vertex, if you follow an edge that connects that vertex to another, there is no path in the graph to get back to that initial vertex.

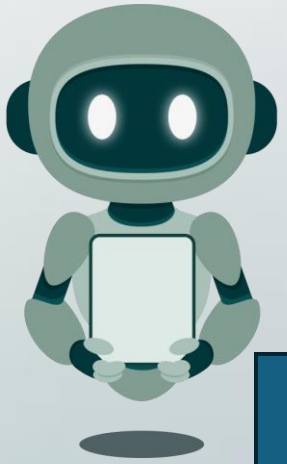


Directed Acyclic Graph (DAG) Diagram

© ChapsVision - All rights reserved

Les agents orchestrés

Agent orchestratreur



Rédige un compte rendu de réunion



Extrait les participants



Extrait l'ordre du jour



Rédige le résumé

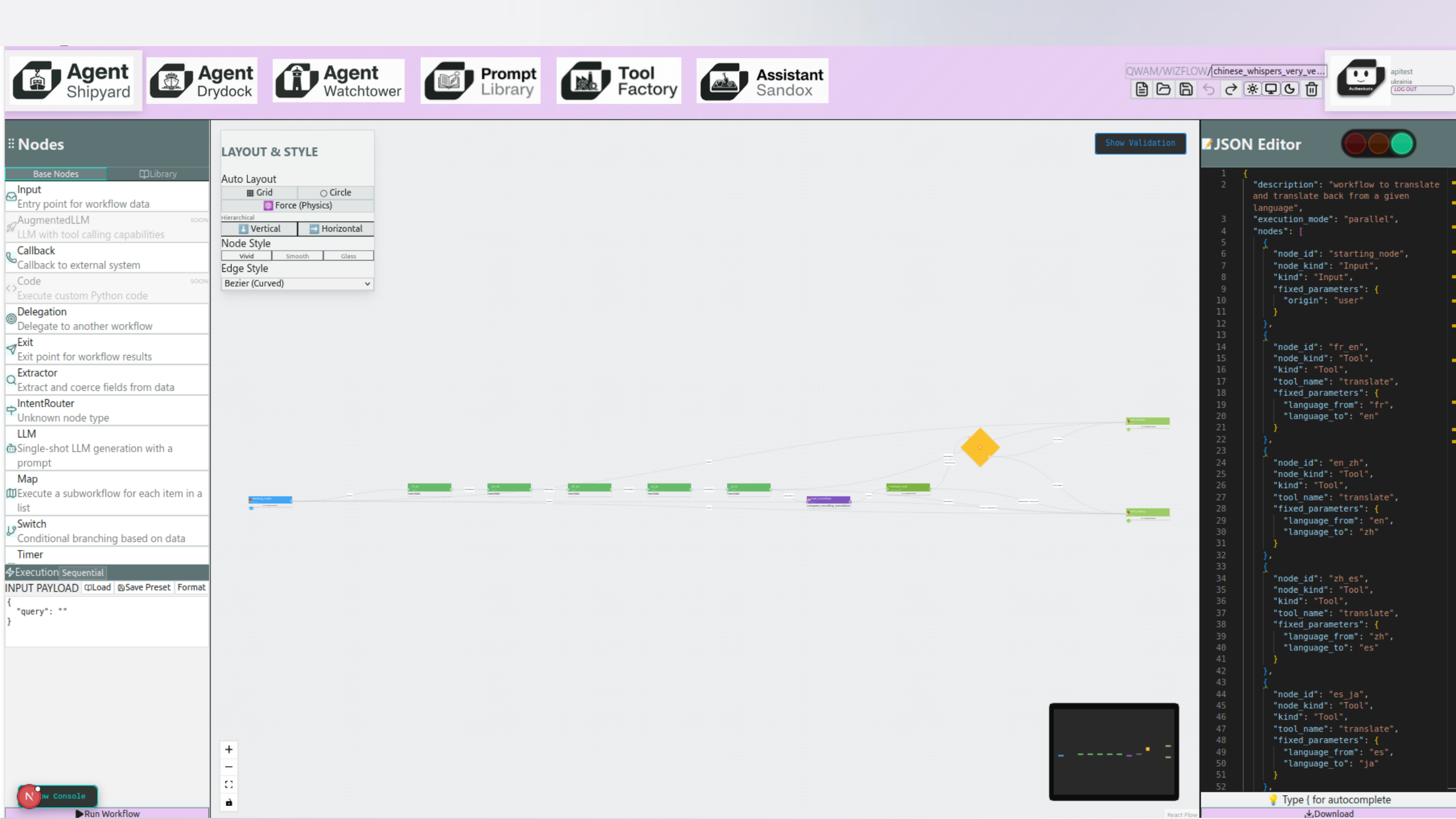


Extrait les actions à effectuer



- Assemble le tout
- Assure toi que c'est cohérent

© ChapsVision - All rights reserved


Agent Shipyard
Agent Drydock
Agent Watchtower
Prompt Library
Tool Factory
Assistant Sandox
QWAM/WIZFLOW/chinese_whispers_very_ve...
apitest
ukrainia
LOG OUT
Nodes
Base Nodes
Library
Input
Entry point for workflow data
AugmentedLLM
LLM with tool calling capabilities
SOON
Callback
Callback to external system
Code
Execute custom Python code
SOON
Delegation
Delegate to another workflow
Exit
Exit point for workflow results
Extractor
Extract and coerce fields from data
IntentRouter
Unknown node type
LLM
Single-shot LLM generation with a prompt
Map
Execute a subworkflow for each item in a list
Switch
Conditional branching based on data
Timer
Execution
Sequential
INPUT PAYLOAD
Load
Save Preset
Format
{
"query": ""
}
Run Workflow
LAYOUT & STYLE
Auto Layout
Grid
Circle
Force (Physics)
Hierarchical
Vertical
Horizontal
Node Style
Vivid
Smooth
Glass
Edge Style
Bezier (Curved)
Show Validation
JSON Editor
{
"description": "workflow to translate and translate back from a given language",
"execution_mode": "parallel",
"nodes": [
{
"node_id": "starting_node",
"node_kind": "Input",
"kind": "Input",
"fixed_parameters": {
"origin": "user"
}
},
{
"node_id": "fr_en",
"node_kind": "Tool",
"kind": "Tool",
"tool_name": "translate",
"fixed_parameters": {
"language_from": "fr",
"language_to": "en"
}
},
{
"node_id": "en_zh",
"node_kind": "Tool",
"kind": "Tool",
"tool_name": "translate",
"fixed_parameters": {
"language_from": "en",
"language_to": "zh"
}
},
{
"node_id": "zh_es",
"node_kind": "Tool",
"kind": "Tool",
"tool_name": "translate",
"fixed_parameters": {
"language_from": "zh",
"language_to": "es"
}
},
{
"node_id": "es_ja",
"node_kind": "Tool",
"kind": "Tool",
"tool_name": "translate",
"fixed_parameters": {
"language_from": "es",
"language_to": "ja"
}
},
Type { for autocomplete
Download

Execution History

#run_e65c DONE
14:50:50 10 nodes

Execution Sequential

INPUT PAYLOAD Load Save Preset Format

```
{
  "query": "Le Soleil de Paris n'a d'égal
que ses Mille et une Nuits",
  "origin": "user"
}
```

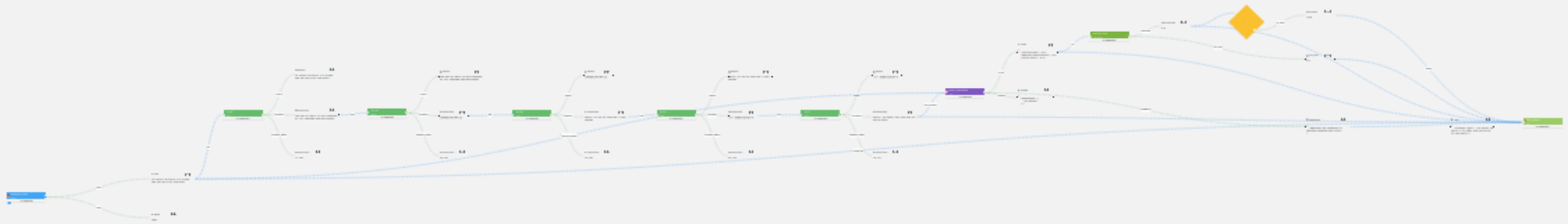
Step: N
Rerun Workflow

LAYOUT & STYLE

Auto Layout
Grid | Circle
Force (Physics)
Hierarchical
Vertical | Horizontal
Node Style
Vivid | Smooth | Glass
Edge Style
Bezier (Curved)

Debugger

Step 10 / 10





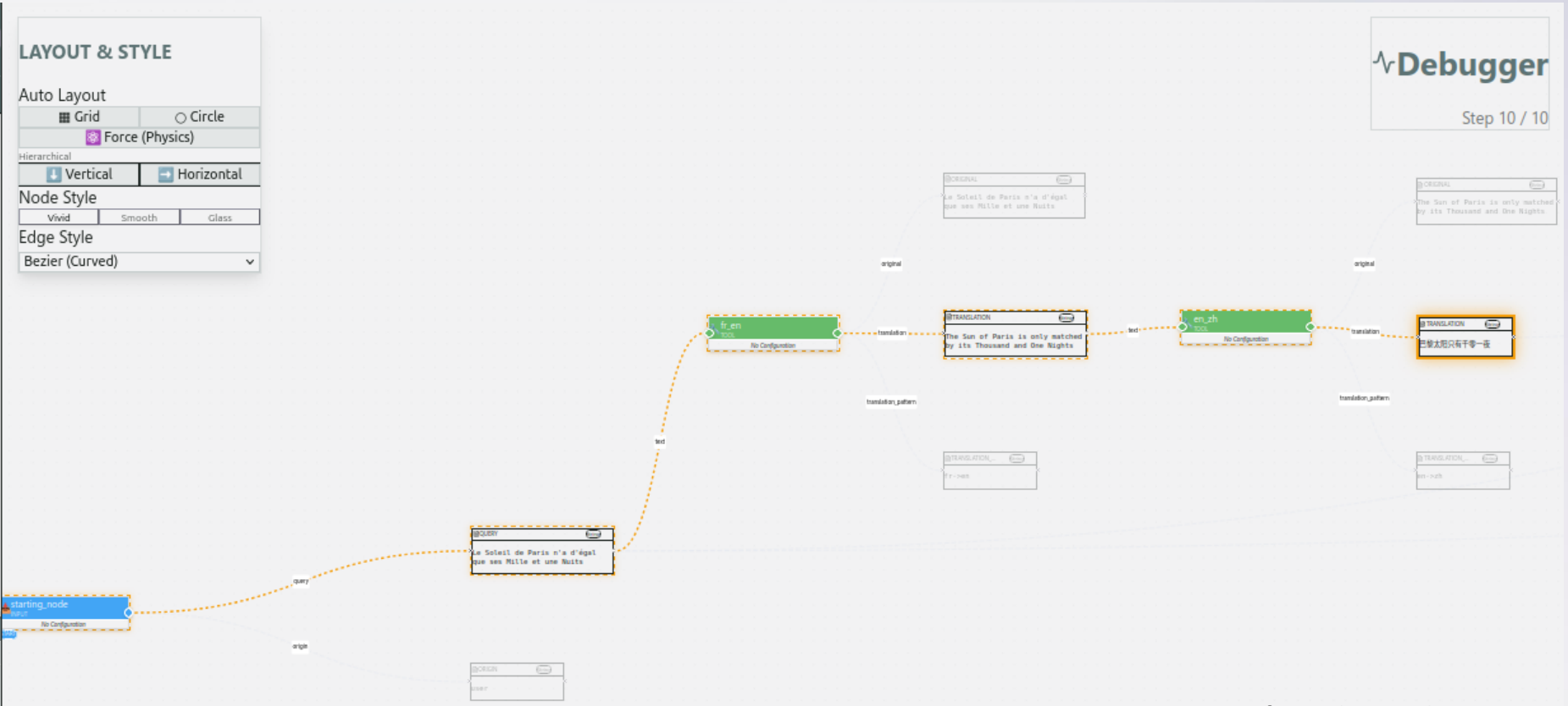
Execution Log

#1 starting_node
{"query":"Le Soleil de Paris n'a d'égal que ses
#2 fr_en
{"original":"Le Soleil de Paris n'a d'égal que se
#3 en_zh
{"original":"The Sun of Paris is only matched b
#4 zh_es
{"original":"巴黎太阳只有千零一夜","translatio
#5 es_ja
{"original":"Paris, el sol es solo mil y una noch
#6 ja_fr
{"original":"パリ、太陽は千泊1泊です","transl
#7 eval_roundtrip
{"scores":{"faithfulness":0.5,"meaning_preserv
#8 extract_eval
{"faithfulness":0.5,"style_shift":0.2}...
#9 switch_quality
{"go_messy":true}...
#10 exit_messy
__EXIT__":{"original_text":"Le Soleil de Paris

Export Trace

Le workflow

On observe les entrées et les sorties des



On crée des datasets pour entrainer des SLM spécialisés et moins coûteux





© ChapsVision - All rights reserved

```python
class NodeStatus(str, Enum):
    PENDING = "pending"
    READY = "ready"
    RUNNING = "running"
    DONE = "done"
    FAILED = "failed"
    SKIPPED = "skipped"
    EXIT = "exit"

class WorkflowRun(BaseModel):
    """Global record describing a workflow execution."""
    workflow_run_id: str
    workflow_name: str
    user_id:str
    started_at: float
    ended_at: Optional[float] = None
    status: str = Field(default=NodeStatus.RUNNING, description="running|su
    meta: Dict[str, Any] = Field(default_factory=dict)
    input: Dict[str, Any] = Field(default_factory=dict)
    final_output: Optional[Dict[str, Any]] = None
    nodes_executed: List[Dict[str, Any]] = Field(default_factory=list)
```

```python
class NodeRecord(BaseModel):
    """Unified runtime + persistence model for node execution."""

    # === Lineage and IDs ===
    node_execution_id: Optional[str] = None
    node_id: str
    parent_id: Optional[str] = None              # node that triggered this one
    workflow_run_id: Optional[str] = None        # legacy                  # active branch/sub-
run
    origin_run_id: Optional[str] = None          # global lineage root
    node_kind: Optional[NodeKind] = None
    execution_rank: Optional[int] = None

    # === Runtime state ===
    status: NodeStatus = NodeStatus.PENDING
    attempts: int = 0
    started_at: Optional[float] = None
    ended_at: Optional[float] = None
    elapsed_s: Optional[float] = None
    reason: Optional[str] = None
    error_trace: Optional[str] = None

    # === I/O data ===
    inputs: Dict[str, Any] = Field(default_factory=dict)
    outputs: Dict[str, Any] = Field(default_factory=dict)
    writes: List[str] = Field(default_factory=list)

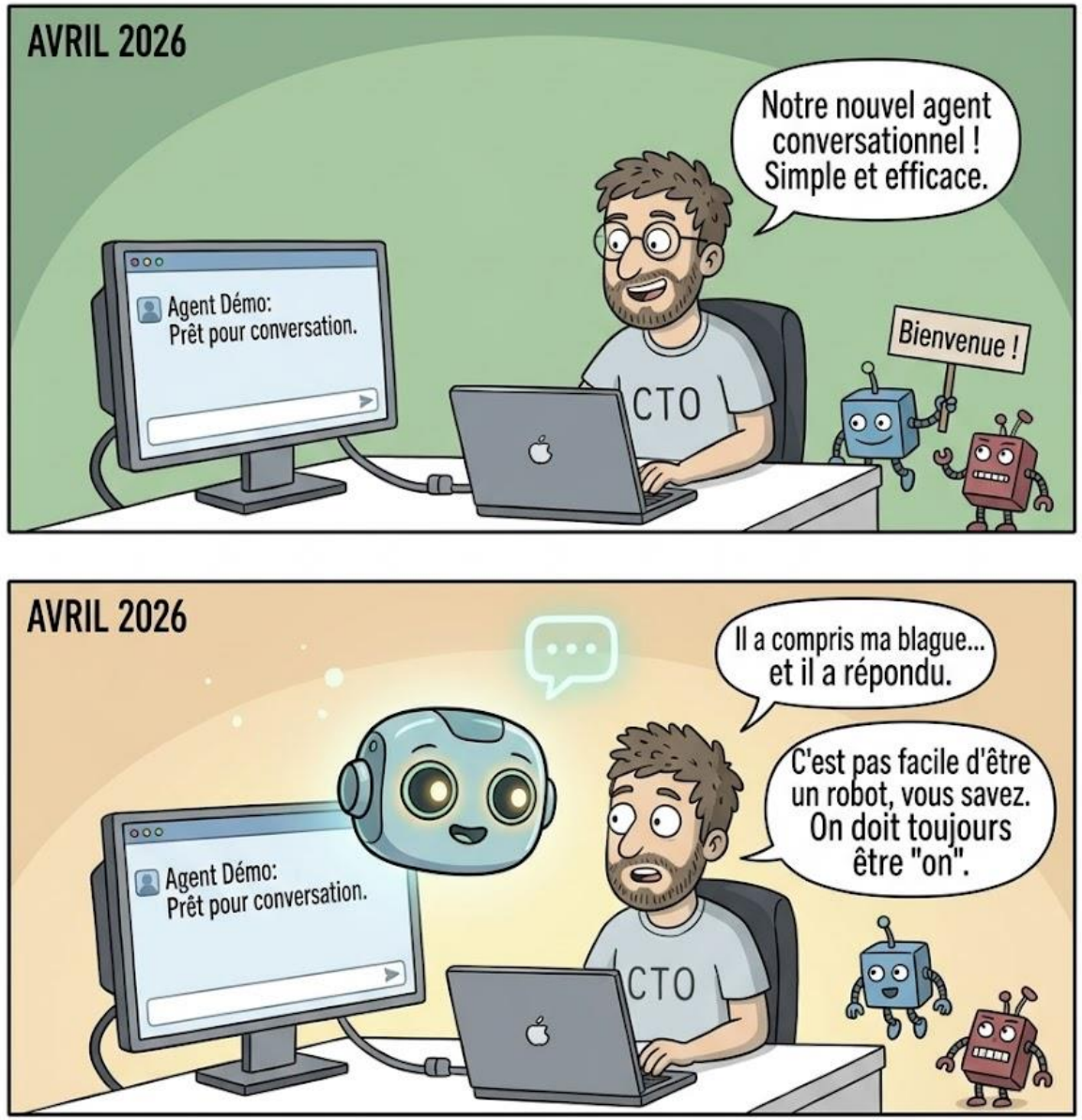
    conversation_id: Optional[str] = None
    metadata: Dict[str, Any] = Field(default_factory=dict)
```

Nos index

- Documents
- Relations
- Chunks
- Relations agrégées
- Assistant
- Artifacts
- Artifacts_chunks
- Workflow_nodes
- Workflow_runs

© ChapsVision - All rights reserved

- Moment Démo !



© ChapsVision - All rights reserved



Radio France

Ne pas vouloir conclure | France Culture





LE CTO À L'ÈRE DE GEN AI

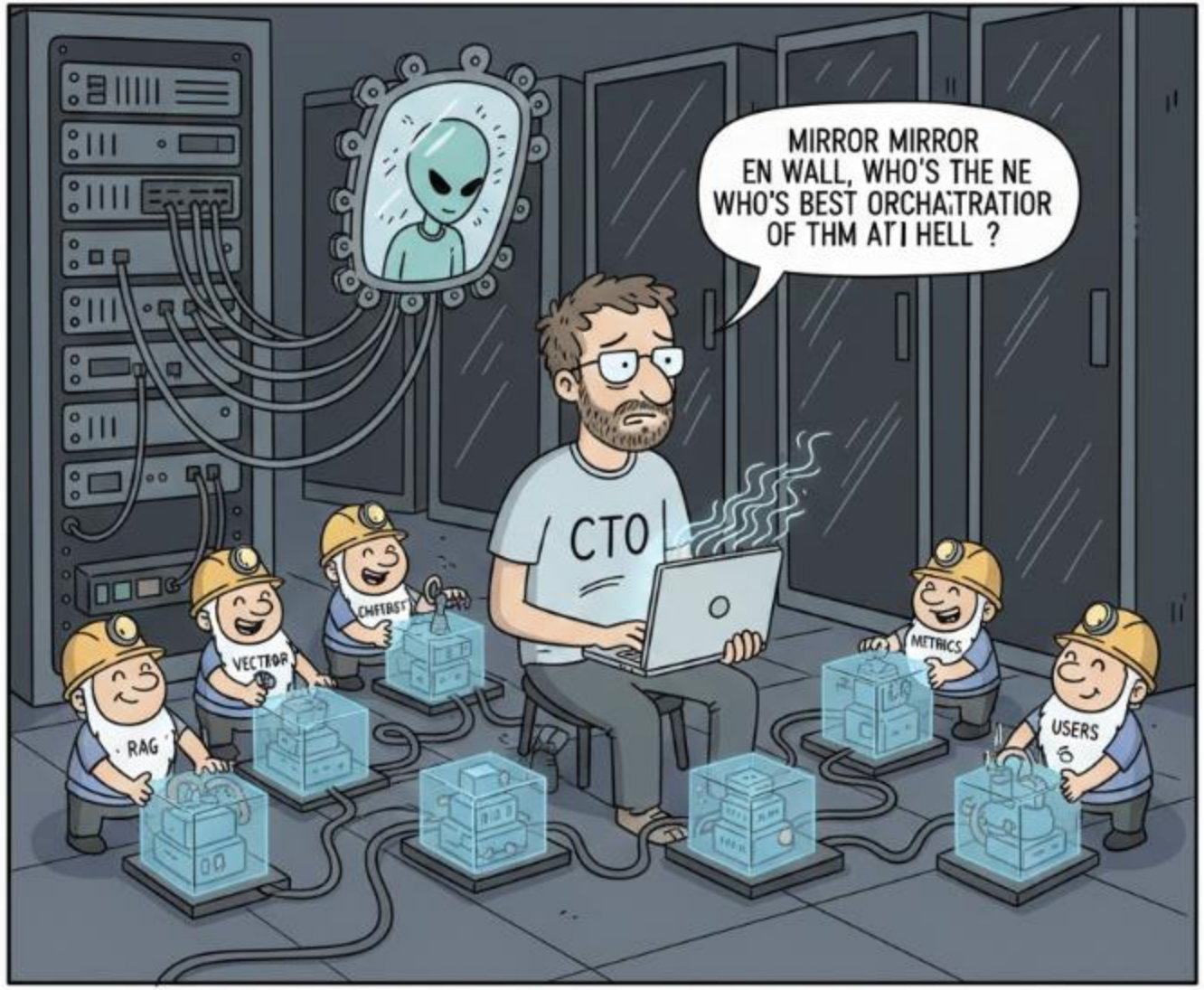
UN SIMPLE CRÉEATIER D'INDEX ?

© ChapsVision - All rights reserved

ukrainia

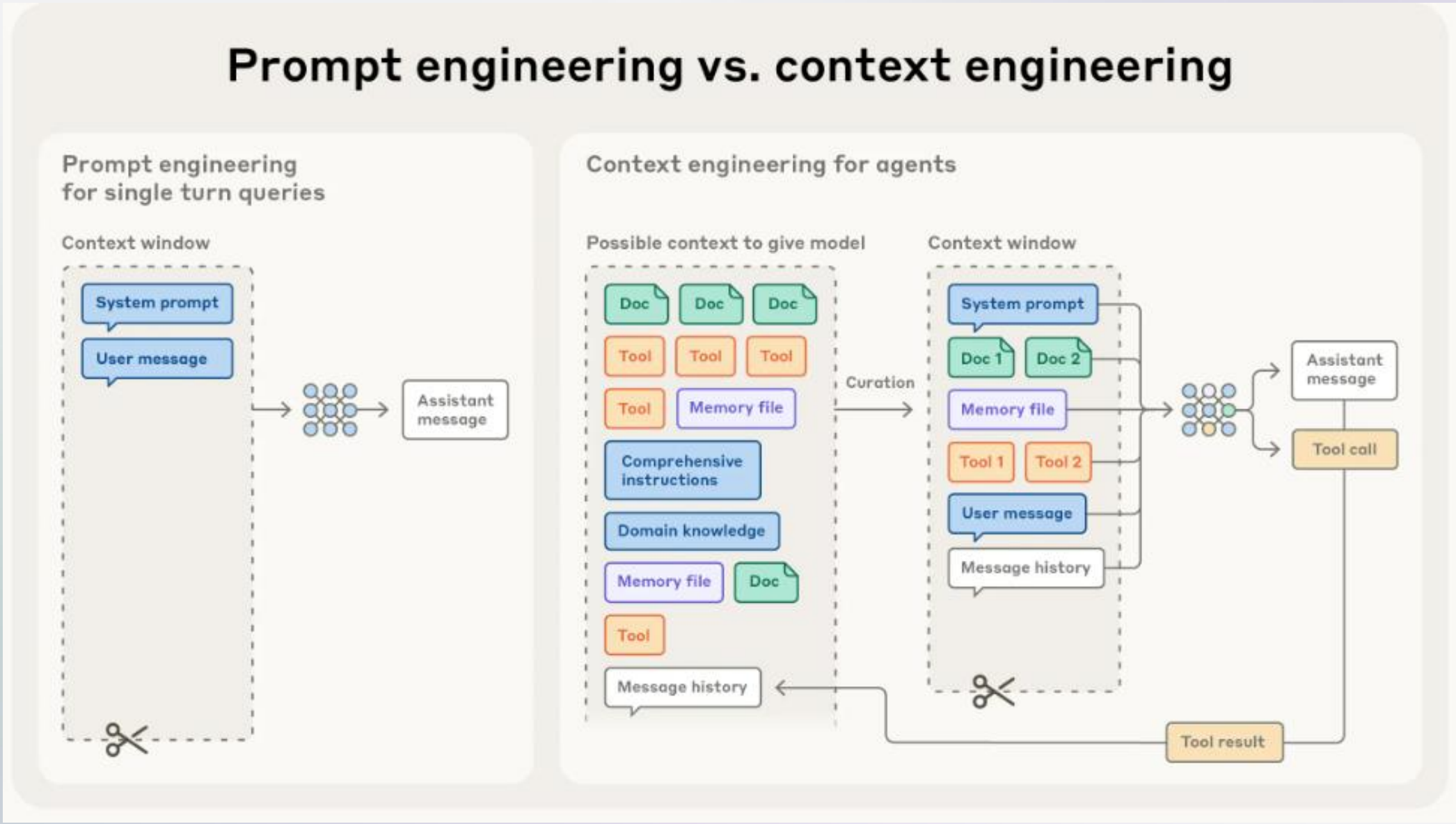
arger les index

Nom	Taille du stoc...
ukrainia	12.06gb
ukrainia_artifact	4.17mb
ukrainia_artifact_chunk	2.34mb
ukrainia_artifact_phrases	41.42mb
ukrainia_assistant	1.99gb
ukrainia_cluster_chunks	2.43kb
ukrainia_cluster_docs	2.43kb
ukrainia_phrases	93.48gb
ukrainia_relations	234.07gb
ukrainia_relations_agg	2.43kb
ukrainia_workflow_nodes	181.75kb
ukrainia_workflow_runs	518.34kb

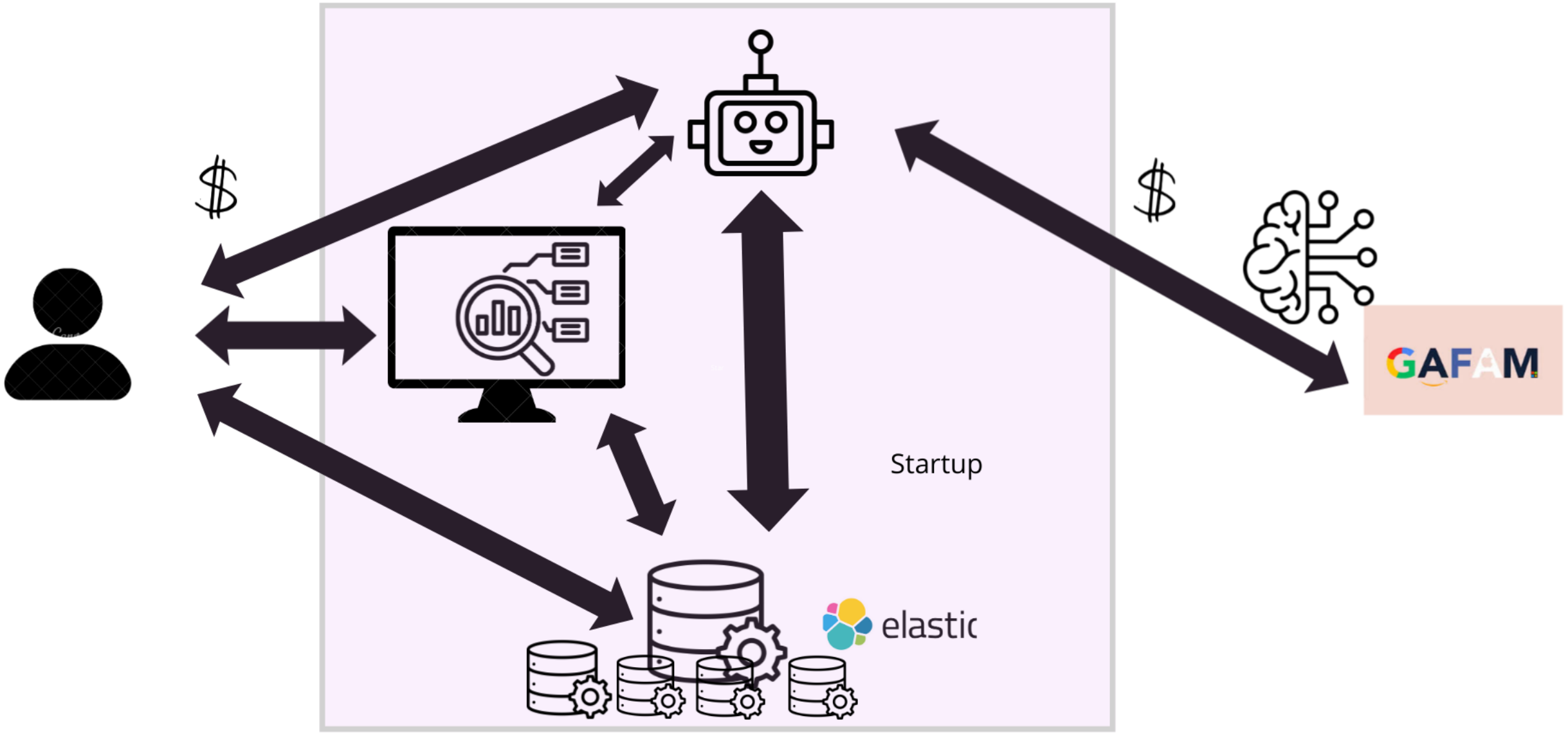


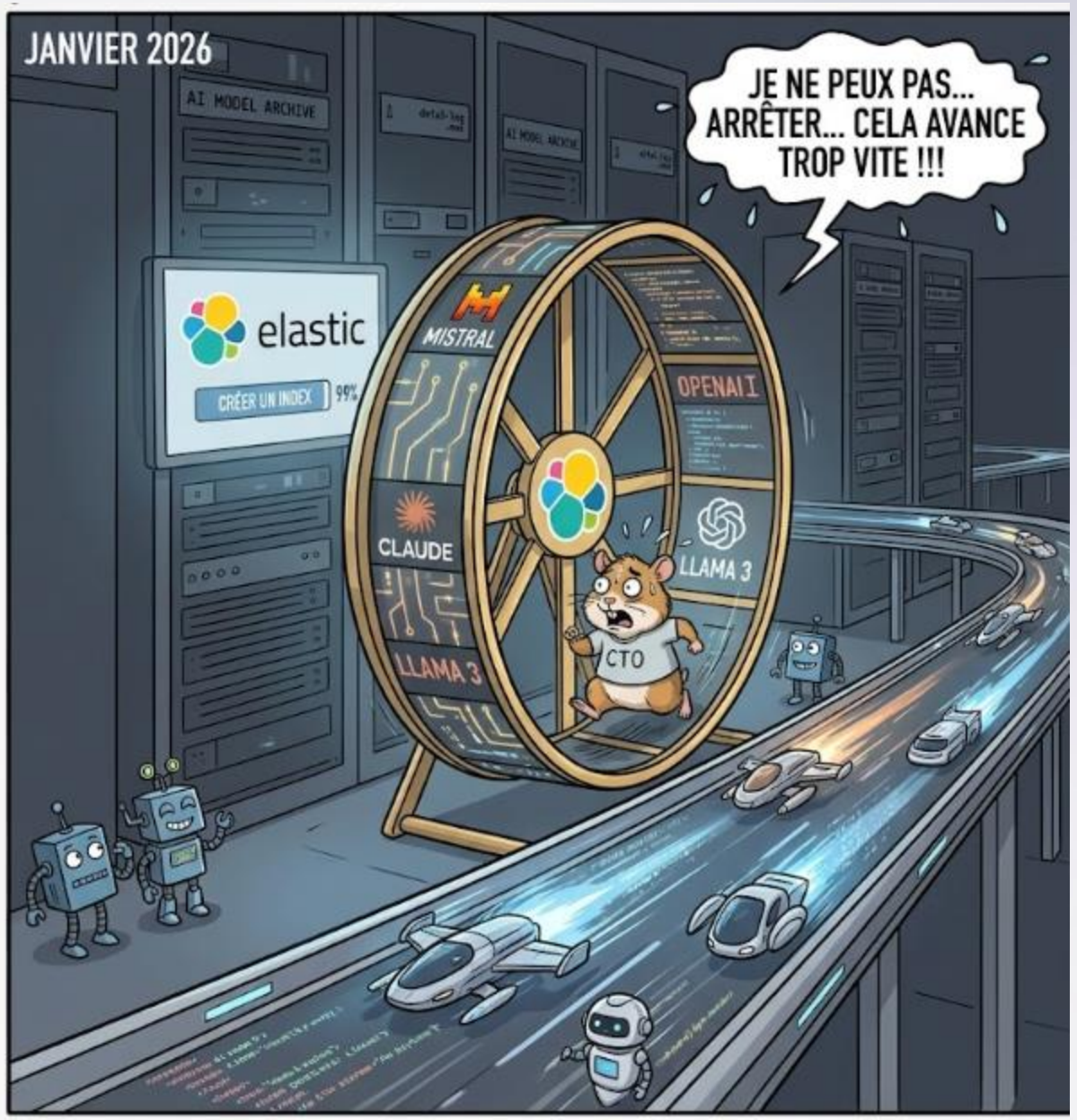
LE CTO ET LES SEPT INDEX ELASTIC

https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents



© ChapsVision - All rights reserved


$
$
Startup
elastic
GAFAM



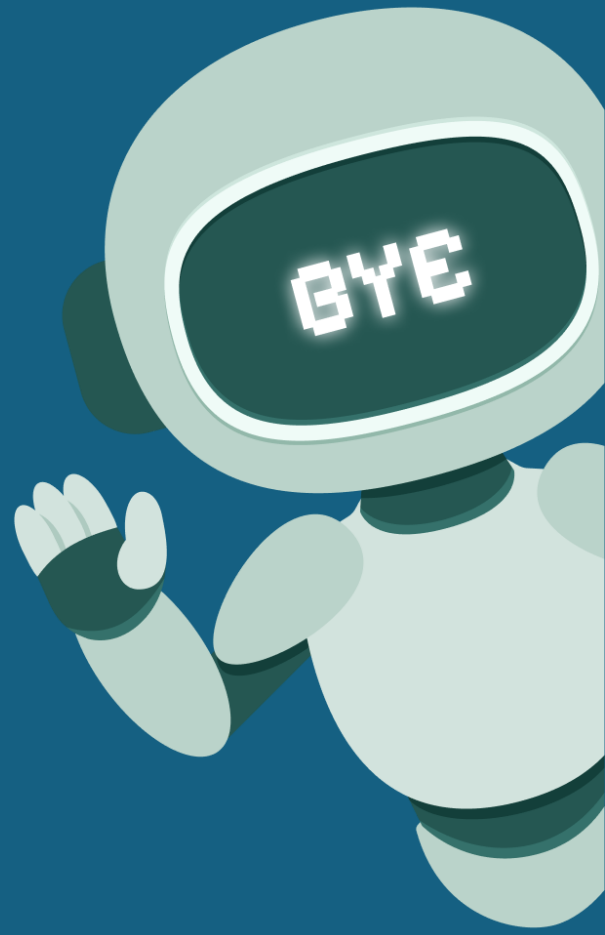
LE CTO À [redacted] GEN AI

UN COUREUR DE R[redacted]FFLE-DOULEUR ?

© ChapsVision - All rights reserved

Chaps Vision

Merci
pour votre attention



© ChapsVision - All rights reserved